

ATS Resume Builder

Complete Implementation Blueprint — Every File, Every Line of Code

Flutter App AI-Powered Watch-Ad-to-Download Full Revenue Strategy

Platform	Flutter 3.x (Dart) — Android + iOS + Web
AI Engine	Google Gemini 2.5 Flash + Groq Fallback
Revenue Model	Watch-Ad-to-Download + Pro Subscription + B2B
MVP Timeline	10–12 weeks (solo) / 6–8 weeks (2-person team)
Prepared	April 2026 — Anthropic / Claude

TABLE OF CONTENTS

PART 1 — PRODUCT & BUSINESS

1. Executive Summary & Market
2. Watch-Ad-to-Download Model & Revenue Projections
3. Full Revenue Strategy (All 6 Streams)
4. Complete Feature Set

PART 2 — ARCHITECTURE

5. Technical Architecture & Stack Decisions
6. AI API Strategy — Gemini + Groq Fallback
7. Database Schema — All 5 Firestore Collections
8. App Screens & Complete User Flow
9. ATS Compliance Rules — 14 Hard Constraints

PART 3 — FULL CODE IMPLEMENTATION

10. pubspec.yaml + Project Structure
11. main.dart + app.dart + app_theme.dart + app_router.dart
12. Firebase Auth Service + Auth Screens
13. Firestore Service (Full CRUD)
14. Firebase Storage Service
15. Dashboard Screen
16. Template Picker Screen
17. Resume Editor — All Sections
18. AI Service — Gemini + Groq Fallback
19. ATS Score Screen
20. JD Matcher Screen
21. Resume Preview Screen
22. PDF Generation Service
23. AdMob Rewarded Ads — Download Screen
24. Banner & Interstitial Ads
25. Cover Letter Builder Screen

- | | |
|-----|--|
| 26. | Job Tracker Screen (Kanban) |
| 27. | Pro Upgrade Screen |
| 28. | Settings Screen |
| 29. | Onboarding Screen |
| 30. | Payment Service — Razorpay + Stripe |
| 31. | Backend API — Node.js/Express (Full) |
| 32. | Firestore Security Rules |
| 33. | AndroidManifest.xml + Info.plist Setup |
| 34. | GitHub Actions CI/CD Workflow |

PART 4 — BUSINESS

- | | |
|-----|--------------------------------------|
| 35. | Development Timeline + Cost Estimate |
| 36. | Competitive Landscape + Go-to-Market |
| 37. | Risks & Mitigation + Quick Reference |

1. Executive Summary & Market

Item	Details
Market Size	\$1.2 Billion globally (2025), growing 8% YoY
Target Users	Fresh graduates, job switchers, freelancers — India & South Asia
Platform	Flutter — single codebase for Android + iOS + Web
AI Engine	Google Gemini 2.5 Flash (free 500 req/day → paid \$0.30/M tokens)
Revenue Model	Watch-Ad-to-Download + Pro Subscription + B2B Licensing
MVP Timeline	10–12 weeks solo / 6–8 weeks 2-person team
Launch Cost	~Rs 0 first year (all free tiers) + Rs 8,750 store fees
Break-even	~500 downloads/day covers Apple subscription cost

Core Value Proposition

- Build a polished, ATS-safe resume in under 5 minutes — no design skills needed.
- AI writes bullet points, summaries, and tailors keywords to any job description.
- Get an ATS score (0–100) with specific, actionable fixes before applying.
- Download the resume as a professional PDF completely FREE — just watch a short ad.
- Manage multiple versions of your resume for different roles or companies.

2. Watch-Ad-to-Download Model & Revenue Projections

Step 1 — Build	User fills in details, picks a template, and the app generates their ATS-optimised resume.
Step 2 — Preview	User sees a full-screen preview. A watermark/blur overlay is applied to the download button area.
Step 3 — Watch Ad	User taps "Download for Free". A rewarded video ad plays (15–30s, cannot be skipped). Countdown timer shown.
Step 4 — Unlock	Ad completes. PDF is generated on-device and saved to the user's phone. No payment ever required.
Step 5 — Re-download	Each new download (e.g., updated version) requires watching another ad. Unlimited edits always free.

Network	RPM (India)	Notes
Google AdMob Rewarded	\$3–15 / 1,000 views	Best Flutter integration, most reliable fill rate
Unity Ads	\$4–12 / 1,000 views	High eCPM for app installs category
IronSource / LevelPlay	\$5–18 / 1,000 views	Mediation — multiple networks, maximises revenue
AppLovin MAX	\$6–20 / 1,000 views	Premium network, requires approval

Recommendation: Start with Google AdMob (easiest Flutter integration) and add IronSource mediation at 1,000+ DAU. Mediation increases revenue by 20–40%.

Daily Downloads	Ad Views/Day	RPM	Daily Revenue	Monthly Revenue
100	100	\$6	\$0.60	\$18
500	500	\$6	\$3.00	\$90
2,000	2,000	\$6	\$12.00	\$360
10,000	10,000	\$8 (mediation)	\$80.00	\$2,400
50,000	50,000	\$10 (premium)	\$500.00	\$15,000

3. Full Revenue Strategy — All 6 Streams

Rewarded Video Ads

PRIMARY

User watches 15–30s ad to download resume PDF. AdMob + IronSource mediation. \$6–20 CPM India, \$20–60 CPM international. No payment needed from user.

Pro Subscription

SECONDARY

Rs 99–199/month India or \$4.99/month international. Unlocks: unlimited ATS scans, premium templates, cover letter builder, LinkedIn optimiser, job tracker, no ads. Freemium converts at 2–5% of active users.

Interstitial / Banner Ads

SUPPLEMENTARY

Shown on non-critical screens (dashboard, template gallery). Lower RPM than rewarded ads but passive revenue. Keep minimal to protect UX.

B2B / College Licensing

HIGH VALUE

Sell bulk access to colleges, placement cells, coding bootcamps, HR firms. Rs 20,000–2,00,000/year per institution. One B2B deal = 10,000 ad views.

Job Board Affiliates

PASSIVE

Partner with Naukri, LinkedIn, Internshala, Indeed. Earn Rs 5–50 per user who clicks a job listing. Add "Jobs matching your resume" section.

Human Review Upsell

PREMIUM

Paid expert review of AI-generated resume. Price Rs 299–999 per review. Partner with freelance HR consultants. High margin.

4. Complete Feature Set

Core Resume Building

Feature	Description
Resume Editor	Section-by-section form: Personal info, Summary, Experience, Education, Skills, Projects, Certifications, Languages, Custom sections
Drag-and-Drop Sections	Reorder sections freely with drag handles. Order saved per resume.
Rich Text Fields	Bold, italic, bullet points inside experience descriptions
Auto-save	Resume saved to cloud every 30 seconds. Never lose work.
Multi-resume	Create unlimited resumes (Free: 2 max, Pro: unlimited). Duplicate and modify.
LinkedIn Import	Enter LinkedIn profile URL → AI scrapes public data to pre-fill fields

ATS Optimisation

Feature	Description
ATS Score Checker	Analyses 24+ criteria: formatting, keywords, section headings, file type. Returns score 0–100.
JD Matcher	Paste any JD → AI compares it to resume and shows missing keywords and skills.
Keyword Suggestions	AI suggests relevant keywords for the target role and industry.
ATS-Safe Templates	All templates: single-column layout, standard fonts, no tables, no graphics, selectable text.
Section Heading Validation	Checks headings match ATS dictionaries (e.g., "Work Experience" not "Career Journey").
File Format Check	Warns if output format (image PDF vs text PDF) will fail ATS parsing.

AI Features

Feature	Description
AI Summary Generator	Generates personalised professional summary from experience and target role.

AI Bullet Point Writer	User types rough job duty → AI rewrites as quantified, action-verb-led bullet point.
AI Rewrite	Highlight any text → tap Improve → AI rewrites it clearer and more impactful.
Cover Letter Builder	Generates tailored cover letter from resume + JD in user's tone.
AI Feedback	After ATS scan, AI explains each issue in plain English and suggests exact fixes.
Skill Gap Analysis	Compares user skills to JD requirements and lists missing skills to learn.

Download & Export

Feature	Description
Watch-Ad PDF Download	User watches rewarded video ad → resume PDF downloaded to device. Free, always.
PDF Quality	Proper text layer (not rasterised image). Selectable, copy-pasteable, ATS-readable.
DOCX Export	Pro feature: download as Word document for further editing.
Share Link	Shareable link to resume (hosted on server, expires 30 days). Free.
Multiple Formats	A4 and US Letter page sizes. Portrait only (ATS standard).

Templates & Design

Feature	Description
Template Library	10 free ATS-safe templates. 5 premium templates (Pro). All single-column.
Colour Themes	8 colour options per template. ATS-compatible (colour only in headings).
Font Selection	4 ATS-safe font families: Calibri-style, Arial-style, Times-style, Garamond-style.
Font Size Control	Body text 10–12pt, headings 13–16pt. ATS-safe range enforced.
Template Preview	Full-screen live preview before downloading.

User Account & Management

Feature	Description
---------	-------------

Auth	Email/password + Google sign-in. One account across Android, iOS, and Web.
Cloud Sync	Resumes synced to Firebase. Access on any device.
Version History	Last 5 versions of each resume saved. Restore any version (Pro).
Job Tracker	Track applications: company, role, status (Applied/Interview/Offer/Rejected). Pro feature.
Dashboard	Shows all resumes with ATS score badge, last edited date, download count.

5. Technical Architecture & Stack Decisions

Layer	Technology	Why / Notes
Mobile App	Flutter 3.x (Dart)	Android + iOS + Web from one codebase
State Management	Riverpod 2.x	Compile-safe providers, code generation, no context needed
AI Integration	Google Gemini API (REST)	Gemini 2.5 Flash — free 500 req/day, paid \$0.30/M tokens
AI Fallback	Groq API (Llama 3)	14,400 req/day free — fastest inference, used when Gemini quota exceeded
Backend API	Node.js + Express	Handles payment verification, rate limiting, server-side AI calls
Database	Firebase Firestore	Free tier: 1GB, 50k reads/day. Real-time sync built-in.
Authentication	Firebase Auth	Google sign-in + email/password. Free up to 10k MAU.
File Storage	Firebase Storage	Store generated PDFs and resume thumbnails. Free: 5GB.
PDF Generation	flutter_html_to_pdf	Client-side HTML→PDF. ATS-safe text layer guaranteed.
Ad Integration	Google AdMob + IronSource	Rewarded ads (primary) + banner/interstitial (supplementary).
Payments India	Razorpay	Best for INR, UPI, cards, net banking.
Payments Global	Stripe	USD/international Pro subscriptions.
Push Notifications	Firebase Cloud Messaging	Free. ATS score alerts, job match notifications.
Analytics	Firebase Analytics + Mixpanel	Firebase free. Mixpanel for funnel/conversion analysis.
Crash Reporting	Firebase Crashlytics	Free. Essential for production stability.
Hosting Backend	Render (free tier)	512MB RAM, 0.5 CPU free. Upgrade to Starter \$7/mo at scale.
CI/CD	GitHub Actions + Fastlane	Auto-build and deploy to Play Store + App Store on push to main.

6. AI API Strategy — Gemini + Groq Fallback

API	Free Tier	Cost (Paid)	Best For
Gemini 2.5 Flash	500 req/day, 15 req/min	\$0.30/M tokens	Primary: all features
Gemini 2.5 Flash-Lite	1,000 req/day	\$0.075/M tokens	High-volume keyword matching
Groq (Llama 3)	14,400 req/day	\$0.05/M tokens	Fallback when Gemini quota exceeded
Mistral API	Limited	\$0.25/M tokens	European users (GDPR compliant)
OpenAI GPT-4o-mini	None	\$0.15/M tokens	Highest quality writing output
Cohere Command R	Limited	\$0.15/M tokens	Keyword extraction specialist

Strategy: Use Gemini free tier for MVP. Add Groq as automatic fallback when Gemini hits 500 req/day limit (most generous free tier available). Move to Gemini paid at scale — costs less than \$3/day at 10,000 AI calls.

AI Prompt Architecture — All 4 Prompts

Feature	System Prompt	User Input
Bullet Writer	Expert resume writer. Rewrite as quantified, action-verb-led bullet. Past tense, under 20 words. Return ONLY the bullet.	Raw job duty text + role context
Summary Generator	Expert resume writer. Write 3-4 sentence professional summary. No "I". Include years of experience and 2-3 skills. Return ONLY the summary.	Name, target role, experience highlights, skills
ATS Score Checker	ATS expert. Analyse resume against 24 criteria. Return JSON: {score, passed, failed, issues, fixes, keywords, missing_keywords}.	Resume text + optional job description
JD Keyword Matcher	Extract top 15 keywords from JD. Check which are in resume. Return JSON: {required_keywords, matched, missing, match_percentage}.	Job description + resume text
Cover Letter	Write professional cover letter, first person, 3 paragraphs, under 300 words. Match resume tone. Do not invent facts. Return ONLY the letter.	Resume text + JD + company name + applicant name

7. Database Schema — All 5 Firestore Collections

users/{userId}

Field	Type	Description
uid	String	Firebase Auth UID
email	String	User email address
name	String	Full name
plan	String	"free" or "pro"
planExpiry	Timestamp	When Pro subscription expires
adsWatched	Integer	Total rewarded ads watched (analytics)
createdAt	Timestamp	Account creation date
lastActive	Timestamp	Last app open

users/{userId}/resumes/{resumeld}

Field	Type	Description
title	String	User-given name e.g. "Software Engineer Resume"
templateId	String	Which template is applied
colorTheme	String	Selected colour theme key
atsScore	Integer	Last ATS score (0–100)
sections	Map	Ordered list of sections with content
targetRole	String	Job title being targeted
targetJD	String	Pasted job description (optional)
lastEdited	Timestamp	Last modification time
downloadCount	Integer	How many times PDF was downloaded
versions	Array	Last 5 snapshots of full resume data (version history)

users/{userId}/coverLetters/{coverId}

Field	Type	Description
resumeld	String	Linked resume ID
content	String	Full cover letter text

targetCompany	String	Company name
generatedAt	Timestamp	When AI generated it

users/{userId}/applications/{appId}

Field	Type	Description
resumeld	String	Resume used for this application
company	String	Company name
role	String	Job title
status	String	"applied" "interview" "offer" "rejected"
appliedAt	Timestamp	Application date
notes	String	User notes

templates/{templateId}

Field	Type	Description
name	String	Template display name
isPremium	Boolean	True = Pro only
thumbnailUrl	String	Preview image URL in Firebase Storage
htmlLayout	String	HTML/CSS template string for PDF generation
atsScore	Integer	Pre-tested ATS score of blank template (all should be 95+)

8. App Screens & Complete User Flow

Screen	Purpose & Key Contents
Onboarding (3 slides)	Shown once on first open. Slide 1: ATS explained. Slide 2: AI features. Slide 3: Free download model. Skip button always visible.
Auth Screen	Google sign-in button + email/password form. "Continue as guest" for limited preview. Forgot password flow.
Dashboard	All user resumes with ATS score badge, last edited date, thumbnail. FAB to create new. Empty state with CTA.
Template Picker	Grid of 15 templates with ATS badge and Premium tag. Filter by style. Tap to preview full-screen.
Resume Editor	Sectioned scrollable form. Each section collapsible. AI Improve button per field. Live ATS score badge at top. Auto-save indicator.
AI Assistant Panel	Slides up from bottom. Chat-style interface for AI rewrites, summaries, and suggestions.
ATS Score Screen	Full-screen score display (0–100 gauge). Issues list with red/amber/green dots. Tap issue to jump to fix in editor.
JD Matcher Screen	Two-panel: paste JD on left, matched/missing keywords on right. One-tap to add missing keywords to resume.
Resume Preview	Full-screen PDF-quality preview with pinch-to-zoom. Download button prominent at bottom.
Download Screen	Watch Ad button (rewarded video trigger). Progress indicator. "Watch 30-second ad to download free." After ad: PDF saved confirmation.
Cover Letter Builder	Generates cover letter from resume + JD. Edit inline. Download via same ad model (free).
Job Tracker (Pro)	Kanban-style board: Applied / Interview / Offer / Rejected columns. Drag cards between columns.
Settings	Account info, subscription status, notification preferences, data export, delete account.
Pro Upgrade Screen	Feature comparison table. Monthly / annual pricing toggle. Razorpay/Stripe payment sheet.

User Flow: Onboarding → Auth → Dashboard → Template Picker → Resume Editor → ATS Score → JD Matcher → Preview → Watch Ad → PDF Downloaded. Full flow: under 7 minutes for a first-time user.

9. ATS Compliance Rules — 14 Hard Constraints

Rule	Implementation	Why It Matters
Single-column layout only	CSS: no multi-column grid, no floats, no absolute positioning	Multi-column layouts confuse ATS parsers — text reads out of order
Standard section headings	Enforce: Work Experience, Education, Skills, Certifications, Projects	ATS dictionaries recognise standard labels only
No tables for layout	Use block elements only. Tables OK for skills grid only.	ATS parsers often skip table content entirely
No headers/footers	All content in main body. Name/contact at top of body.	PDF headers and footers frequently missed by parsers
No images or icons	Text-only templates. No profile photos, no icon fonts.	Images are invisible to ATS — waste of space
Selectable text PDF	Generate PDF with text layer, not rasterised image	Image PDFs score 0 — ATS cannot read them
Standard fonts only	Helvetica, Times-Roman, Courier — embedded in PDF	Non-standard fonts cause character encoding errors
Font size 10–12pt body	Enforce min 10pt, max 12pt for body text	Sub-10pt text may not be extracted correctly
No special characters	Strip/replace: smart quotes, em-dashes → plain equivalents	Encoding issues cause ATS parsing failures
Keyword density 1–3%	AI checker flags keyword stuffing or absence	Too few = rejected. Too many = flagged as spam.
Bullet points UTF-8 •	Use standard bullet (U+2022), not custom shapes	Custom bullet characters may not parse correctly
No colour in body text	Colour only in headings/name. Body text: #000000	Coloured body text sometimes fails extraction
File size under 2MB	Compress PDF on generation. Target <500KB.	Large files sometimes fail upload to ATS portals
Dates in MM/YYYY format	Validate and format all dates on save	Inconsistent date formats confuse tenure calculation

10. pubspec.yaml + Project Structure

pubspec.yaml

```
name: ats_resume_builder
description: AI-powered ATS resume builder with watch-ad-to-download model
version: 1.0.0+1
publish_to: none

environment:
  sdk: '>=3.2.0 <4.0.0'
  flutter: '>=3.16.0'

dependencies:
  flutter:
    sdk: flutter
  # AI
  google_generative_ai: ^0.4.0
  http: ^1.2.0
  # Firebase
  firebase_core: ^2.24.0
  firebase_auth: ^4.15.0
  cloud_firestore: ^4.13.0
  firebase_storage: ^11.5.0
  firebase_messaging: ^14.7.0
  firebase_analytics: ^10.7.0
  firebase_crashlytics: ^3.4.0
  google_sign_in: ^6.2.0
  # Ads
  google_mobile_ads: ^4.0.0
  # PDF
  flutter_html_to_pdf: ^0.6.0
  printing: ^5.11.0
  path_provider: ^2.1.1
  # State Management
  flutter_riverpod: ^2.4.9
  riverpod_annotation: ^2.3.3
  # Payments
  razorpay_flutter: ^1.3.5
  flutter_stripe: ^10.0.0
  # UI
  google_fonts: ^6.1.0
  reorderables: ^0.6.0
  percent_indicator: ^4.2.3
  flutter_animate: ^4.3.0
```



```

cached_network_image: ^3.3.0
shimmer: ^3.0.0
# Utils
uuid: ^4.2.1
intl: ^0.19.0
shared_preferences: ^2.2.2
go_router: ^12.1.1
url_launcher: ^6.2.2
share_plus: ^7.2.1
image_picker: ^1.0.4
dev_dependencies:
flutter_test:
sdk: flutter
flutter_lints: ^3.0.0
build_runner: ^2.4.7
riverpod_generator: ^2.3.9
custom_lint: ^0.5.7
riverpod_lint: ^2.3.7

```

Project Folder Structure

```

lib/
  main.dart
  app.dart
  firebase_options.dart # Generated by: flutterfire configure
  core/
    constants/
      app_colors.dart
      app_strings.dart
      app_dimensions.dart
    theme/app_theme.dart
    router/app_router.dart
    utils/
      validators.dart
      date_formatter.dart
    models/
      resume_model.dart
      user_model.dart
      template_model.dart
      application_model.dart
    services/
      ai_service.dart # Gemini + Groq fallback
      auth_service.dart # Firebase Auth
      firestore_service.dart # Full CRUD
      storage_service.dart # Firebase Storage

```

```
■ ■■■ pdf_service.dart # HTML-to-PDF generation
■ ■■■ admob_service.dart # Rewarded + banner + interstitial ads
■ ■■■ payment_service.dart # Razorpay + Stripe
■■■ providers/
■ ■■■ auth_provider.dart
■ ■■■ resume_provider.dart
■ ■■■ ai_provider.dart
■ ■■■ ad_provider.dart
■■■ features/
■■■ onboarding/onboarding_screen.dart
■■■ auth/
■ ■■■ login_screen.dart
■ ■■■ signup_screen.dart
■■■ dashboard/dashboard_screen.dart
■■■ template_picker/template_picker_screen.dart
■■■ resume_editor/
■ ■■■ resume_editor_screen.dart
■ ■■■ sections/
■ ■■■ personal_info_section.dart
■ ■■■ experience_section.dart
■ ■■■ education_section.dart
■ ■■■ skills_section.dart
■■■ ats_checker/ats_score_screen.dart
■■■ jd_matcher/jd_matcher_screen.dart
■■■ preview/
■ ■■■ resume_preview_screen.dart
■ ■■■ download_screen.dart
■■■ cover_letter/cover_letter_screen.dart
■■■ job_tracker/job_tracker_screen.dart
■■■ pro/pro_upgrade_screen.dart
■■■ settings/settings_screen.dart
```

11. main.dart + app.dart + app_theme.dart + app_router.dart

lib/main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_crashlytics/firebase_crashlytics.dart';
import 'package:google_mobile_ads/google_mobile_ads.dart';
import 'app.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  await MobileAds.instance.initialize();
  // Catch all Flutter errors and send to Crashlytics
  FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;
  runApp(const ProviderScope(child: ATSResumeApp()));
}
```

lib/app.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'core/router/app_router.dart';
import 'core/theme/app_theme.dart';

class ATSResumeApp extends ConsumerWidget {
  const ATSResumeApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(routerProvider);
    return MaterialApp.router(
      title: 'ATS Resume Builder',
      theme: AppTheme.lightTheme,
      darkTheme: AppTheme.darkTheme,
      themeMode: ThemeMode.system,
      routerConfig: router,
      debugShowCheckedModeBanner: false,
    );
  }
}
```

lib/core/theme/app_theme.dart

```
import 'package:flutter/material.dart';
```

```

import 'package:google_fonts/google_fonts.dart';

class AppTheme {
  static const Color primary = Color(0xFF6366F1); // Indigo
  static const Color secondary = Color(0xFF10B981); // Emerald
  static const Color surface = Color(0xFFFF1F5F9);
  static const Color error = Color(0xFFEF4444);

  static ThemeData get lightTheme => ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: primary, brightness: Brightness.light),
    textTheme: GoogleFonts.interTextTheme(),
    appBarTheme: const AppBarTheme(
      backgroundColor: Colors.white, elevation: 0,
      foregroundColor: Color(0xFF1E293B),
      titleTextStyle: TextStyle(color: Color(0xFF1E293B),
        fontSize: 18, fontWeight: FontWeight.w600)),
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
        backgroundColor: primary, foregroundColor: Colors.white,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
        padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 14))),
    cardTheme: CardTheme(
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: const BorderSide(color: Color(0xFFE2E8F0))),
      color: Colors.white),
  );

  static ThemeData get darkTheme => ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: primary, brightness: Brightness.dark),
    textTheme: GoogleFonts.interTextTheme(ThemeData.dark().textTheme),
  );
}

```

lib/core/router/app_router.dart

```

import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../features/onboarding/onboarding_screen.dart';
import '../features/auth/login_screen.dart';
import '../features/dashboard/dashboard_screen.dart';
import '../features/template_picker/template_picker_screen.dart';
import '../features/resume_editor/resume_editor_screen.dart';

```

```

import '../features/ats_checker/ats_score_screen.dart';
import '../features/jd_matcher/jd_matcher_screen.dart';
import '../features/preview/resume_preview_screen.dart';
import '../features/preview/download_screen.dart';
import '../features/cover_letter/cover_letter_screen.dart';
import '../features/job_tracker/job_tracker_screen.dart';
import '../features/pro/pro_upgrade_screen.dart';
import '../features/settings/settings_screen.dart';
import '../providers/auth_provider.dart';

final routerProvider = Provider<GoRouter>((ref) {
  final authState = ref.watch(authStateProvider);
  return GoRouter(
    initialLocation: '/onboarding',
    redirect: (ctx, state) {
      final isLoggedIn = authState.value != null;
      final isAuthRoute = state.uri.path == '/login' ||
        state.uri.path == '/onboarding';
      if (!isLoggedIn && !isAuthRoute) return '/login';
      if (isLoggedIn && isAuthRoute) return '/dashboard';
      return null;
    },
    routes: [
      GoRoute(path: '/onboarding', builder: (_, __) => const OnboardingScreen()),
      GoRoute(path: '/login', builder: (_, __) => const LoginScreen()),
      GoRoute(path: '/dashboard', builder: (_, __) => const DashboardScreen()),
      GoRoute(path: '/templates', builder: (_, __) => const TemplatePickerScreen()),
      GoRoute(path: '/editor/:id', builder: (_, s) =>
        ResumeEditorScreen(resumeId: s.pathParameters['id']!)),
      GoRoute(path: '/ats/:id', builder: (_, s) =>
        ATSScoreScreen(resumeId: s.pathParameters['id']!)),
      GoRoute(path: '/jd/:id', builder: (_, s) =>
        JDMatcherScreen(resumeId: s.pathParameters['id']!)),
      GoRoute(path: '/preview/:id', builder: (_, s) =>
        ResumePreviewScreen(resumeId: s.pathParameters['id']!)),
      GoRoute(path: '/download/:id', builder: (_, s) =>
        DownloadScreen(resumeId: s.pathParameters['id']!)),
      GoRoute(path: '/cover-letter/:id', builder: (_, s) =>
        CoverLetterScreen(resumeId: s.pathParameters['id']!)),
      GoRoute(path: '/job-tracker', builder: (_, __) => const JobTrackerScreen()),
      GoRoute(path: '/pro', builder: (_, __) => const ProUpgradeScreen()),
      GoRoute(path: '/settings', builder: (_, __) => const SettingsScreen()),
    ],
  );
});

```

12. Firebase Auth Service + Auth Screens

```
lib/services/auth_service.dart
```

```
import 'package:firebase_auth/firebase_auth.dart';
import 'package:google_sign_in/google_sign_in.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
```

```
final authServiceProvider = Provider<AuthService>((ref) => AuthService());
final authstateProvider = StreamProvider<User?>((ref) =>
  FirebaseAuth.instance.authStateChanges());
```

```
class AuthService {
    final _auth = FirebaseAuth.instance;
    final _db = FirebaseFirestore.instance;
    final _googleSignIn = GoogleSignIn();
```

```
User? get currentUser => _auth.currentUser;
```

[illegible]

```
Future<UserCredential> signUpWithEmail(String email, String password,  
String name) async {
```

```
final cred = await _auth.createUserWithEmailAndPassword(
```

```
email: email, password: password);
```

```
await cred.user?.updateDisplayName(name);
```

```
await _createUserDoc(cred.user!, name);
```

```
return cred;
```

} _____

[illegible]

```
Future<UserCredential> signInWithEmail(String email, String password) async {
```

```
return _auth.signInWithEmailAndPassword(email: email, password: password);
```

} _____

[illegible]

```
Future<UserCredential?> signInWithGoogle() async {
```

```
final googleUser = await _googleSignIn.signIn();
```

```
if (googleUser == null) return null;
```

```
final googleAuth = await googleUser.authentication;
```

```
final credential = GoogleAuthProvider.credential(
```

```
accessToken: googleAuth.accessToken, idToken: googleAuth.idToken);
```

```
final cred = await auth.signInWithCredential(credential);
```

```
// Create user doc if first time
```

```
final userDoc = await db.collection('users').doc(cred.user!.uid).get();
```

```
final UserDoc = await _db.collection( 'users' ).doc(cred.user!.uid).get();
if (UserDoc.exists) {
```

```
visit: createNewDoc(createdUser!, createdUser!, displayName, 0, 1);
```

```
await _createUserDoc(cred.user!, cred.user!.displayName ?? "");
```

[illegible]

lib/features/auth/login_screen.dart

```
class _LoginScreenState extends ConsumerState<LoginScreen> {
  final _emailCtrl = TextEditingController();
  final _passCtrl = TextEditingController();
  final _formKey = GlobalKey<FormState>();
}
```

```

bool _loading = false;
bool _isSignUp = false;
final _nameCtrl = TextEditingController();

Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => _loading = true);
  try {
    final auth = ref.read(authServiceProvider);
    if (_isSignUp) {
      await auth.signUpWithEmail(_emailCtrl.text.trim(),
        _passCtrl.text.trim(), _nameCtrl.text.trim());
    } else {
      await auth.signInWithEmail(_emailCtrl.text.trim(), _passCtrl.text.trim());
    }
    if (mounted) context.go('/dashboard');
  } on Exception catch (e) {
    if (mounted) ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(e.toString()), backgroundColor: Colors.red));
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

Future<void> _googleSignIn() async {
  setState(() => _loading = true);
  try {
    final cred = await ref.read(authServiceProvider).signInWithGoogle();
    if (cred != null && mounted) context.go('/dashboard');
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(24),
        child: Form(
          key: _formKey,
          child: Column(crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const SizedBox(height: 48),
              const Text('ATS Resume Builder',
                style: TextStyle(fontSize: 26, fontWeight: FontWeight.bold),

```



```

textAlign: TextAlign.center),
const SizedBox(height: 8),
Text(_isSignUp ? 'Create your account' : 'Welcome back',
style: const TextStyle(color: Colors.grey),
textAlign: TextAlign.center),
const SizedBox(height: 40),
if (_isSignUp)
TextFormField(controller: _nameCtrl,
decoration: const InputDecoration(labelText: 'Full Name',
border: OutlineInputBorder()),
validator: (v) => (v?.isEmpty ?? true) ? 'Enter your name' : null),
if (_isSignUp) const SizedBox(height: 16),
TextFormField(controller: _emailCtrl, keyboardType: TextInputType.emailAddress,
decoration: const InputDecoration(labelText: 'Email',
border: OutlineInputBorder()),
validator: (v) => (v?.contains('@') ?? false) ? null : 'Enter valid email'),
const SizedBox(height: 16),
TextFormField(controller: _passCtrl, obscureText: true,
decoration: const InputDecoration(labelText: 'Password',
border: OutlineInputBorder()),
validator: (v) => (v?.length ?? 0) >= 6 ? null : 'Min 6 characters'),
const SizedBox(height: 24),
_loading
? const Center(child: CircularProgressIndicator())
: ElevatedButton(onPressed: _submit,
child: Text(_isSignUp ? 'Create Account' : 'Sign In')),
const SizedBox(height: 16),
OutlinedButton.icon(onPressed: _googleSignIn,
icon: const Icon(Icons.g_mobiledata),
label: const Text('Continue with Google')),
TextButton(
onPressed: () => setState(() => _isSignUp = !_isSignUp),
child: Text(_isSignUp
? 'Already have an account? Sign In'
: "Don't have an account? Sign Up")),
]),
),
),
),
);
}
}

```

13. Firestore Service — Full CRUD

```
lib/services/firestore_service.dart
```

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../models/resume_model.dart';

final firestoreServiceProvider = Provider<FirestoreService>((ref) =>
FirestoreService());

class FirestoreService {
  final _db = FirebaseFirestore.instance;

  // ■■ USER ████████████████████████████████████████████████████████████████████████████
  Stream<Map<String,dynamic?>> userStream(String uid) =>
    _db.collection('users').doc(uid).snapshots()
      .map((s) => s.data());

  Future<Map<String,dynamic?>> getUser(String uid) async {
    final doc = await _db.collection('users').doc(uid).get();
    return doc.data();
  }

  Future<void> updateUser(String uid, Map<String,dynamic> data) =>
    _db.collection('users').doc(uid).update(data);

  // ■■ RESUMES ████████████████████████████████████████████████████████████████████████████
  Stream<List<ResumeModel>> resumesStream(String uid) =>
    _db.collection('users').doc(uid).collection('resumes')
      .orderBy('lastEdited', descending: true)
        .snapshots()
          .map((s) => s.docs.map(ResumeModel.fromFirestore).toList());

  Stream<ResumeModel> resumeStream(String uid, String resumeId) =>
    _db.collection('users').doc(uid).collection('resumes').doc(resumeId)
      .snapshots()
        .map((s) => ResumeModel.fromFirestore(s));

  Future<String> createResume(String uid, ResumeModel resume) async {
    final ref = await _db.collection('users').doc(uid)
      .collection('resumes').add(resume.toFirestore());
    return ref.id;
  }

  Future<void> saveResume(String uid, ResumeModel resume) =>
    _db.collection('users').doc(uid).collection('resumes').doc(resume.id)
      .set(resume.toFirestore(), SetOptions(merge: true));

  Future<void> deleteResume(String uid, String resumeId) =>
    _db.collection('users').doc(uid).collection('resumes').doc(resumeId).delete();
}
```

[illegible]

[illegible]

14. Firebase Storage Service

```
lib/services/storage_service.dart
```

[illegible]

```
final task = ref.putFile(file);  
await for (final snap in task.snapshotEvents) {  
  yield snap.bytesTransferred / snap.totalBytes;  
}  
}  
}
```

15. Dashboard Screen

lib/features/dashboard/dashboard_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../providers/resume_provider.dart';
import '../providers/auth_provider.dart';
import '../models/resume_model.dart';
import '../services/admob_service.dart';

class DashboardScreen extends ConsumerWidget {
  const DashboardScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final resumes = ref.watch(resumeListProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text('My Resumes'),
        actions: [
          IconButton(icon: const Icon(Icons.settings),
            onPressed: () => context.push('/settings'),
          ),
        ],
      ),
      body: resumes.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => Center(child: Text('Error: $e')),
        data: (list) => list.isEmpty
          ? _buildEmptyState(context)
          : ListView.builder(
              padding: const EdgeInsets.all(16),
              itemCount: list.length,
              itemBuilder: (ctx, i) => _ResumeCard(resume: list[i]),
            ),
        floatingActionButton: FloatingActionButton.extended(
          onPressed: () => context.push('/templates'),
          icon: const Icon(Icons.add),
          label: const Text('New Resume'),
        ),
        bottomNavigationBar: _buildBannerAd(ref),
      );
  }

  Widget _buildEmptyState(BuildContext context) => Center(
```

```

child: Column(mainAxisAlignment: MainAxisAlignment.center, children: [
const Icon(Icons.description_outlined, size: 72, color: Colors.grey),
const SizedBox(height: 16),
const Text('No resumes yet', style: TextStyle(fontSize: 20, fontWeight:
FontWeight.bold)),
const SizedBox(height: 8),
const Text('Create your first ATS-optimised resume', style: TextStyle(color:
Colors.grey)),
const SizedBox(height: 24),
ElevatedButton(onPressed: () => context.push('/templates'),
child: const Text('Create Resume')),
]),
);

Widget _buildBannerAd(WidgetRef ref) {
final adWidget = ref.watch(bannerAdProvider);
return adWidget ?? const SizedBox(height: 50);
}

}

class _ResumeCard extends ConsumerWidget {
final ResumeModel resume;
const _ResumeCard({required this.resume});

Color _scoreColor(int score) => score >= 80 ? Colors.green
: score >= 60 ? Colors.orange : Colors.red;

@override
Widget build(BuildContext context, WidgetRef ref) {
return Card(
margin: const EdgeInsets.only(bottom: 12),
child: ListTile(
contentPadding: const EdgeInsets.all(16),
title: Text(resume.title,
style: const TextStyle(fontWeight: FontWeight.bold)),
subtitle: Column(crossAxisAlignment: CrossAxisAlignment.start,
children: [
const SizedBox(height: 4),
Text('Last edited: ${_formatDate(resume.lastEdited)}',
style: const TextStyle(color: Colors.grey, fontSize: 12)),
Text('Downloads: ${resume.downloadCount}',
style: const TextStyle(color: Colors.grey, fontSize: 12)),
]),
trailing: Column(mainAxisAlignment: MainAxisAlignment.center,
children: [
Container(
padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
decoration: BoxDecoration(

```



```

color: _scoreColor(resume.atsScore).withOpacity(0.1),
borderRadius: BorderRadius.circular(20),
border: Border.all(color: _scoreColor(resume.atsScore))),
child: Text('${resume.atsScore}',
style: TextStyle(color: _scoreColor(resume.atsScore),
fontWeight: FontWeight.bold)),
),
const SizedBox(height: 4),
const Text('ATS', style: TextStyle(fontSize: 10, color: Colors.grey)),
]),
onTap: () => context.push('/editor/${resume.id}'),
onLongPress: () => _showOptions(context, ref),
),
);
}

void _showOptions(BuildContext ctx, WidgetRef ref) {
showModalBottomSheet(context: ctx, builder: (_) => SafeArea(
child: Column(mainAxisSize: MainAxisSize.min, children: [
ListTile(leading: const Icon(Icons.visibility),
title: const Text('Preview'),
onTap: () { Navigator.pop(ctx); ctx.push('/preview/${resume.id}'); }),
ListTile(leading: const Icon(Icons.download),
title: const Text('Download'),
onTap: () { Navigator.pop(ctx); ctx.push('/download/${resume.id}'); }),
ListTile(leading: const Icon(Icons.delete, color: Colors.red),
title: const Text('Delete', style: TextStyle(color: Colors.red)),
onTap: () {
Navigator.pop(ctx);
ref.read(resumeActionsProvider).deleteResume(resume.id);
}),
]),
));
}

String _formatDate(DateTime dt) {
final diff = DateTime.now().difference(dt);
if (diff.inDays == 0) return 'Today';
if (diff.inDays == 1) return 'Yesterday';
return '\${dt.day}/\${dt.month}/\${dt.year}';
}
}

```

16. Template Picker Screen

lib/features/template_picker/template_picker_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../providers/auth_provider.dart';

const List<Map<String, dynamic>> kTemplates = [
  {'id': 'classic', 'name': 'Classic', 'premium': false, 'ats': 98, 'color': 0xFF475569},
  {'id': 'modern', 'name': 'Modern', 'premium': false, 'ats': 97, 'color': 0xFF6366F1},
  {'id': 'clean', 'name': 'Clean', 'premium': false, 'ats': 99, 'color': 0xFF059669},
  {'id': 'professional', 'name': 'Professional', 'premium': false, 'ats': 96, 'color': 0xFF2563EB},
  {'id': 'minimal', 'name': 'Minimal', 'premium': false, 'ats': 98, 'color': 0xFF11827},
  {'id': 'executive', 'name': 'Executive', 'premium': false, 'ats': 97, 'color': 0xFF7C3AED},
  {'id': 'tech', 'name': 'Tech', 'premium': false, 'ats': 96, 'color': 0xFF0369A1},
  {'id': 'creative_safe', 'name': 'Creative Safe', 'premium': false, 'ats': 95, 'color': 0xFFEA580C},
  {'id': 'academic', 'name': 'Academic', 'premium': false, 'ats': 98, 'color': 0xFF166534},
  {'id': 'simple', 'name': 'Simple', 'premium': false, 'ats': 99, 'color': 0xFF374151},
  {'id': 'pro_elite', 'name': 'Elite', 'premium': true, 'ats': 99, 'color': 0xFF6366F1},
  {'id': 'pro_bold', 'name': 'Bold', 'premium': true, 'ats': 97, 'color': 0xFFDC2626},
  {'id': 'pro_ivy', 'name': 'Ivy League', 'premium': true, 'ats': 98, 'color': 0xFF1E3A5F},
  {'id': 'pro_startup', 'name': 'Startup', 'premium': true, 'ats': 96, 'color': 0xFF7C3AED},
  {'id': 'pro_global', 'name': 'Global', 'premium': true, 'ats': 97, 'color': 0xFF0F766E},
];

class TemplatePickerScreen extends ConsumerStatefulWidget {
  const TemplatePickerScreen({super.key});

  @override
  ConsumerState<TemplatePickerScreen> createState() => _TemplatePickerScreenState();
}

class _TemplatePickerScreenState extends ConsumerState<TemplatePickerScreen> {
  String? _selectedId;

  void _select(Map<String, dynamic> template, bool isPro) {
    if (template['premium'] == true && !isPro) {
      // Show Pro upgrade prompt
      showDialog(context: context, builder: (_) => AlertDialog(
        title: const Text('Pro Template'),
        content: const Text('This template is available with Pro subscription.'),
      ),
    );
  }
}
```

```

actions: [
  TextButton(onPressed: () => Navigator.pop(context), child: const Text('Cancel')),
  ElevatedButton(onPressed: () {
    Navigator.pop(context); context.push('/pro');
  }, child: const Text('Upgrade to Pro')),
],
));
return;
}
setState(() => _selectedId = template['id']);
}

```

```

@override
Widget build(BuildContext context) {
  final user = ref.watch(userDataProvider).value;
  final isPro = user?['plan'] == 'pro';

  return Scaffold(
    appBar: AppBar(title: const Text('Choose Template')),
    actions: [
      if (_selectedId != null)
        TextButton(onPressed: () => context.push('/editor/new?template=$_selectedId'),
          child: const Text('Use This')),
    ],
    body: GridView.builder(
      padding: const EdgeInsets.all(16),
      gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
        crossAxisCount: 2, crossAxisSpacing: 12, mainAxisSpacing: 12,
        childAspectRatio: 0.72),
      itemCount: kTemplates.length,
      itemBuilder: (ctx, i) {
        final t = kTemplates[i];
        final isSelected = _selectedId == t['id'];
        return GestureDetector(
          onTap: () => _select(t, isPro),
          child: AnimatedContainer(
            duration: const Duration(milliseconds: 200),
            decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(12),
              border: Border.all(
                color: isSelected ? const Color(0xFF6366F1) : Colors.grey.shade200,
                width: isSelected ? 3 : 1),
              boxShadow: isSelected ? [BoxShadow(
                color: const Color(0xFF6366F1).withOpacity(0.3),
                blurRadius: 12)] : null),
            child: Column(children: [

```

```

Expanded(
  child: Container(
    decoration: BoxDecoration(
      color: Color(t['color']).withOpacity(0.1),
      borderRadius: const BorderRadius.vertical(top: Radius.circular(11))),
    child: Center(child: Icon(Icons.description,
      size: 48, color: Color(t['color']))),
    ),
  ),
  Padding(
    padding: const EdgeInsets.all(10),
    child: Row(mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(t['name'], style: const TextStyle(fontWeight: FontWeight.bold)),
        Row(children: [
          if (t['premium'] == true)
            Container(padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
              decoration: BoxDecoration(color: Colors.amber,
                borderRadius: BorderRadius.circular(4)),
              child: const Text('PRO', style: TextStyle(fontSize: 9,
                fontWeight: FontWeight.bold))),
          if (t['premium'] != true) ...[
            const Icon(Icons.verified, size: 12, color: Colors.green),
            const SizedBox(width: 2),
            Text('ATS ${t['ats']}', style: const TextStyle(fontSize: 10, color: Colors.green)),
          ],
        ]),
      ]),
    ),
  ),
);
}},
);
}
}

```

17. Resume Editor — All Sections

lib/features/resume_editor/resume_editor_screen.dart

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../providers/resume_provider.dart';
import 'sections/personal_info_section.dart';
import 'sections/experience_section.dart';
import 'sections/education_section.dart';
import 'sections/skills_section.dart';

class ResumeEditorScreen extends ConsumerStatefulWidget {
  final String resumeId;
  const ResumeEditorScreen({super.key, required this.resumeId});
  @override
  ConsumerState<ResumeEditorScreen> createState() => _State();
}

class _State extends ConsumerState<ResumeEditorScreen> {
  late Timer _autoSaveTimer;
  bool _dirty = false;
  String _saveStatus = 'Saved';

  @override
  void initState() {
    super.initState();
    _autoSaveTimer = Timer.periodic(const Duration(seconds: 30), (_) {
      if (_dirty) _save();
    });
  }

  @override
  void dispose() {
    _autoSaveTimer.cancel();
    if (_dirty) _save();
    super.dispose();
  }

  Future<void> _save() async {
    setState(() => _saveStatus = 'Saving...');
    await ref.read(resumeNotifierProvider(widget.resumeId).notifier).save();
    setState(() { _dirty = false; _saveStatus = 'Saved'; });
  }

  void _onChanged() => setState(() { _dirty = true; _saveStatus = 'Unsaved'; });
}
```

```

@override
Widget build(BuildContext context) {
  final resumeAsync = ref.watch(resumeStreamProvider(widget.resumeId));
  return Scaffold(
    appBar: AppBar(
      title: resumeAsync.whenData((r) => r.title).value != null
        ? Text(resumeAsync.value!.title) : const Text('Edit Resume'),
      actions: [
        Text(_saveStatus, style: TextStyle(
          color: _dirty ? Colors.orange : Colors.green, fontSize: 12)),
        IconButton(icon: const Icon(Icons.save), onPressed: _save),
        IconButton(icon: const Icon(Icons.visibility),
          onPressed: () => context.push('/preview/${widget.resumeId}')),
        PopupMenuButton(itemBuilder: (_) => [
          PopupMenuItem(value: 'ats', child: const Text('Check ATS Score')),
          PopupMenuItem(value: 'jd', child: const Text('Match Job Description')),
          PopupMenuItem(value: 'cover', child: const Text('Generate Cover Letter')),
        ], onSelect: (val) {
          if (val == 'ats') context.push('/ats/${widget.resumeId}');
          if (val == 'jd') context.push('/jd/${widget.resumeId}');
          if (val == 'cover') context.push('/cover-letter/${widget.resumeId}');
        })),
      ],
    ),
    body: resumeAsync.when(
      loading: () => const Center(child: CircularProgressIndicator()),
      error: (e, _) => Center(child: Text('$e')),
      data: (resume) => ListView(
        padding: const EdgeInsets.all(16),
        children: [
          PersonalInfoSection(
            data: Map<String,dynamic>.from(resume.sections['personal'] ?? {}),
            onChanged: (d) {
              ref.read(resumeNotifierProvider(widget.resumeId).notifier)
                .updateSection('personal', d);
            },
            _onChanged(),
          ),
          const SizedBox(height: 12),
          ExperienceSection(
            data: List<Map<String,dynamic>>.from(resume.sections['experience'] ?? []),
            targetRole: resume.targetRole,
            onChanged: (d) {
              ref.read(resumeNotifierProvider(widget.resumeId).notifier)
                .updateSection('experience', d);
            },
          ),
        ],
      ),
    ),
  );
}

```

```

    _onChanged();
  )),
  const SizedBox(height: 12),
  EducationSection(
    data: List<Map<String,dynamic>>.from(resume.sections['education'] ?? []),
    onChanged: (d) {
      ref.read(resumeNotifierProvider(widget.resumeId).notifier)
        .updateSection('education', d);
      _onChanged();
    )),
  const SizedBox(height: 12),
  SkillsSection(
    data: List<String>.from(resume.sections['skills'] ?? []),
    onChanged: (d) {
      ref.read(resumeNotifierProvider(widget.resumeId).notifier)
        .updateSection('skills', d);
      _onChanged();
    )),
  const SizedBox(height: 80),
  ),
  floatingActionButton: FloatingActionButton.extended(
    onPressed: () => context.push('/ats/${widget.resumeId}'),
    icon: const Icon(Icons.analytics),
    label: const Text('ATS Score'),
  ),
);
}
}

```

lib/features/resume_editor/sections/experience_section.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/ai_provider.dart';

class ExperienceSection extends ConsumerStatefulWidget {
  final List<Map<String,dynamic>> data;
  final String targetRole;
  final ValueChanged<List<Map<String,dynamic>>> onChanged;
  const ExperienceSection({super.key, required this.data,
    required this.targetRole, required this.onChanged});
  @override
  ConsumerState<ExperienceSection> createState() => _ExpState();
}

class _ExpState extends ConsumerState<ExperienceSection> {

```

```

late List<Map<String,dynamic>> _items;

@override
void initState() {
  super.initState();
  _items = List.from(widget.data);
}

void _addEntry() {
  setState(() {
    _items.add({'title': '', 'company': '', 'location': '',
'dates': '', 'description': ''});
  });
  widget.onChangeed(_items);
}

void _removeEntry(int i) {
  setState(() => _items.removeAt(i));
  widget.onChangeed(_items);
}

void _update(int i, String key, String value) {
  _items[i][key] = value;
  widget.onChangeed(_items);
}

Future<void> _improveWithAI(int i) async {
  final raw = _items[i]['description'] ?? '';
  if (raw.isEmpty) return;
  final improved = await ref.read(aiServiceProvider)
.improveBullet(raw, widget.targetRole);
  setState(() => _items[i]['description'] = improved);
  widget.onChangeed(_items);
}

@override
Widget build(BuildContext context) {
  return Card(
    child: ExpansionTile(
      title: const Text('Work Experience',
style: TextStyle(fontWeight: FontWeight.bold)),
      initiallyExpanded: true,
      children: [
        ..._items.asMap().entries.map((e) {
          final i = e.key;
          return Padding(
            padding: const EdgeInsets.all(12),
            child: Column(children: [
              Row(children: [

```



```

Expanded(child: TextFormField(
  initialValue: _items[i]['title'],
  decoration: const InputDecoration(labelText: 'Job Title',
    border: OutlineInputBorder()),
  onChanged: (v) => _update(i, 'title', v)),
  const SizedBox(width: 8),
  IconButton(icon: const Icon(Icons.delete, color: Colors.red),
    onPressed: () => _removeEntry(i)),
]),
const SizedBox(height: 8),
TextFormField(initialValue: _items[i]['company'],
  decoration: const InputDecoration(labelText: 'Company',
    border: OutlineInputBorder()),
  onChanged: (v) => _update(i, 'company', v)),
  const SizedBox(height: 8),
  Row(children: [
    Expanded(child: TextFormField(initialValue: _items[i]['location'],
      decoration: const InputDecoration(labelText: 'Location',
        border: OutlineInputBorder()),
      onChanged: (v) => _update(i, 'location', v)),
      const SizedBox(width: 8),
      Expanded(child: TextFormField(initialValue: _items[i]['dates'],
        decoration: const InputDecoration(labelText: 'Dates (MM/YYYY)',
          border: OutlineInputBorder()),
          onChanged: (v) => _update(i, 'dates', v))),
        ],
        const SizedBox(height: 8),
        TextFormField(
          initialValue: _items[i]['description'],
          decoration: InputDecoration(
            labelText: 'Description (bullet points)',
            border: const OutlineInputBorder(),
            suffixIcon: IconButton(
              icon: const Icon(Icons.auto_awesome, color: Color(0xFF6366F1)),
              tooltip: 'Improve with AI',
              onPressed: () => _improveWithAI(i)),
            maxLines: 4,
            onChanged: (v) => _update(i, 'description', v)),
          const Divider(height: 24),
        ]),
      );
    }},
    Padding(
      padding: const EdgeInsets.all(12),

```

```
child: OutlinedButton.icon(onPressed: _addEntry,  
icon: const Icon(Icons.add), label: const Text('Add Experience'))),  
],  
,  
);  
}  
}
```

18. AI Service — Gemini + Groq Fallback

```
lib/services/ai_service.dart
```

[illegible]

```
Future<String> _generateGroq(String prompt) async {  
    final response = await http.post(  
        Uri.parse('https://api.groq.com/openai/v1/chat/completions'),  
        headers: {'Authorization': 'Bearer $_groqKey',  
            'Content-Type': 'application/json'},  
        body: jsonEncode({  
            'model': 'llama3-8b-8192',  
            'messages': [{'role': 'user', 'content': prompt}],  
            'max_tokens': 1024, 'temperature': 0.7,  
        })),  
    );  
  
    if (response.statusCode != 200) throw Exception('Groq error: ${response.body}');  
    final data = jsonDecode(response.body);  
    return data['choices'][0]['message']['content']?.trim() ?? '';  
}  
  
// ■■ Bullet Point Writer ████████████████████████████████████████████████  
Future<String> improveBullet(String rawDuty, String role) async {  
    final prompt =  
        'You are an expert resume writer. Rewrite as a strong, quantified, '  
        'action-verb-led bullet point. Past tense. Under 20 words. '  
        'Role: \${role}. Duty: \${rawDuty}. Return ONLY the bullet.';  
    return _generate(prompt);  
}  
  
// ■■ Professional Summary ████████████████████████████████████████████████  
Future<String> generateSummary({required String name,  
    required String targetRole, required List<String> experiences,  
    required List<String> skills}) async {  
    final prompt = 'Write a 3-4 sentence professional resume summary for \$name '  
        'targeting role \$targetRole. Experience: ${experiences.join(", ")}. '  
        'Skills: ${skills.join(", ")}. No "I". Professional tone. Return ONLY the summary.';  
    return _generate(prompt);  
}  
  
// ■■ ATS Score Checker ██████████████████████████████████████████████████  
Future<ATSResult> checkATS(String resumeText, {String? targetJD}) async {  
    final jd = targetJD != null ? 'Target JD: \$targetJD' : '';  
    final prompt = '''Analyse this resume as an ATS expert against 24 criteria.  
Return ONLY valid JSON (no markdown):  
{  
  "score":0-100,"issues":["string"],"fixes":["string"],  
  "keywords":["string"],"missing_keywords":["string"]  
}  
Resume: \$resumeText \$jd''';  
    try {  
        final raw = await _generate(prompt);  
        final clean = raw.replaceAll(RegExp(r'```[a-z]*'), '').replaceAll('`','').trim();  
        return ATSResult.fromJson(jsonDecode(clean));  
    }
```

[illegible]

```

final List<String> issues, fixes, keywords, missingKeywords;
ATSResult({required this.score, required this.issues, required this.fixes,
required this.keywords, required this.missingKeywords});
factory ATSResult.fromJson(Map<String,dynamic> j) => ATSResult(
score: j['score'] ?? 0,
issues: List<String>.from(j['issues'] ?? []),
fixes: List<String>.from(j['fixes'] ?? []),
keywords: List<String>.from(j['keywords'] ?? []),
missingKeywords: List<String>.from(j['missing_keywords'] ?? []));
}

class KeywordMatchResult {
final List<String> requiredKeywords, matched, missing;
final int matchPercentage;
KeywordMatchResult({required this.requiredKeywords, required this.matched,
required this.missing, required this.matchPercentage});
factory KeywordMatchResult.fromJson(Map<String,dynamic> j) => KeywordMatchResult(
requiredKeywords: List<String>.from(j['required_keywords'] ?? []),
matched: List<String>.from(j['matched'] ?? []),
missing: List<String>.from(j['missing'] ?? []),
matchPercentage: j['match_percentage'] ?? 0);
}

```

19. ATS Score Screen + 20. JD Matcher Screen

lib/features/ats_checker/ats_score_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:percent_indicator/percent_indicator.dart';
import '../services/ai_service.dart';
import '../providers/resume_provider.dart';

class ATSScoreScreen extends ConsumerStatefulWidget {
  final String resumeId;
  const ATSScoreScreen({super.key, required this.resumeId});
  @override
  ConsumerState<ATSScoreScreen> createState() => _ATSSState();
}

class _ATSSState extends ConsumerState<ATSScoreScreen> {
  ATSResult? _result;
  bool _loading = true;
  String? _error;

  @override
  void initState() { super.initState(); WidgetsBinding.instance.addPostFrameCallback((_) => _run()); }

  Future<void> _run() async {
    setState(() { _loading = true; _error = null; });
    try {
      final resume = ref.read(resumeStreamProvider(widget.resumeId)).value;
      if (resume == null) throw Exception('Resume not loaded');
      final text = _serialize(resume);
      final result = await ref.read(aiServiceProvider).checkATS(text,
        targetJD: resume.targetJD.isNotEmpty ? resume.targetJD : null);
      await
        ref.read(resumeNotifierProvider(widget.resumeId).notifier).updateATSScore(result.score);
      setState(() { _result = result; _loading = false; });
    } catch (e) { setState(() { _error = '\$e'; _loading = false; }); }
  }

  String _serialize(resume) {
    final buf = StringBuffer();
    final p = resume.sections['personal'] ?? {};
    buf.writeln('\$p['name'] ?? ''
    \$p['email'] ?? ''
    \$p['phone'] ?? ''');
    buf.writeln('PROFESSIONAL SUMMARY
    \$p['summary'] ?? ''');
```

```

buf.writeln('WORK EXPERIENCE');
for (final e in (resume.sections['experience'] as List? ?? []))
buf.writeln('\${e['title']} at \${e['company']} (\${e['dates']})
\${e['description']}');
buf.writeln('EDUCATION');
for (final e in (resume.sections['education'] as List? ?? []))
buf.writeln('\${e['degree']} - \${e['institution']} (\${e['year']}');
buf.writeln('SKILLS
\${(resume.sections['skills'] as List? ?? []).join(', ')}');
return buf.toString();
}

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('ATS Score'),
    actions: [IconButton(icon: const Icon(Icons.refresh), onPressed: _run)],
    body: _loading
  ? const Center(child: Column(mainAxisAlignment: MainAxisAlignment.center, children: [
    CircularProgressIndicator(), SizedBox(height: 16), Text('Analysing your resume with
    AI...')]))
  : _error != null
  ? Center(child: Column(children: [Text('Error: \$_error'),
    ElevatedButton(onPressed: _run, child: const Text('Retry'))]))
  : _buildResults(),
);

Widget _buildResults() {
  if (_result == null) return const SizedBox();
  final score = _result!.score;
  final col = score >= 80 ? Colors.green : score >= 60 ? Colors.orange : Colors.red;
  return SingleChildScrollView(padding: const EdgeInsets.all(20), children: [
    Center(child: CircularPercentIndicator(radius: 80, lineWidth: 12, percent: score/100,
    center: Column(mainAxisAlignment: MainAxisAlignment.center, children: [
    Text('\$score', style: TextStyle(fontSize: 36, fontWeight: FontWeight.bold, color:
    col)),
    const Text('ATS Score')]),
    progressColor: col, backgroundColor: Colors.grey.shade200,
    circularStrokeCap: CircularStrokeCap.round)),
    const SizedBox(height: 20),
    Container(padding: const EdgeInsets.all(14),
    decoration: BoxDecoration(color: col.withOpacity(0.1),
    borderRadius: BorderRadius.circular(12), border: Border.all(color:
    col.withOpacity(0.3))),
    child: Text(score >= 80 ? 'Excellent – Ready to Apply!' : score >= 60
    ? 'Good – Minor Fixes Needed' : 'Needs Work – Fix Issues First',
    style: TextStyle(color: col, fontWeight: FontWeight.bold, fontSize: 15),
    textAlign: TextAlign.center)),

```



```

if (_result!.issues.isNotEmpty) ...[
const SizedBox(height: 20),
const Text('Issues Found', style: TextStyle(fontSize: 17, fontWeight: FontWeight.bold)),
const SizedBox(height: 10),
..._result!.issues.asMap().entries.map((e) => Card(margin: const EdgeInsets.only(bottom:
8),
child: ListTile(leading: const Icon(Icons.warning_amber, color: Colors.orange),
title: Text(e.value, style: const TextStyle(fontSize: 13)),
subtitle: _result!.fixes.length > e.key
? Text('Fix: \$_result!.fixes[e.key]$',
style: const TextStyle(color: Colors.green, fontSize: 12)) : null))),
],
if (_result!.missingKeywords.isNotEmpty) ...[
const SizedBox(height: 20),
const Text('Missing Keywords', style: TextStyle(fontSize: 17, fontWeight:
FontWeight.bold)),
const SizedBox(height: 10),
Wrap(spacing: 8, runSpacing: 8,
children: _result!.missingKeywords.map((k) => Chip(label: Text(k),
backgroundColor: Colors.red.shade50,
labelStyle: TextStyle(color: Colors.red.shade700))).toList()),
],
]);
}
}

```

lib/features/jd_matcher/jd_matcher_screen.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../services/ai_service.dart';
import '../providers/resume_provider.dart';

class JDMatcherScreen extends ConsumerStatefulWidget {
final String resumeId;
const JDMatcherScreen({super.key, required this.resumeId});
@override
ConsumerState<JDMatcherScreen> createState() => _JDState();
}

class _JDState extends ConsumerState<JDMatcherScreen> {
final _jdCtrl = TextEditingController();
KeywordMatchResult? _result;
bool _loading = false;

Future<void> _analyse() async {
if (_jdCtrl.text.isEmpty) return;
setState(() => _loading = true);

```

```

try {
final resume = ref.read(resumeStreamProvider(widget.resumeId)).value;
if (resume == null) return;
// Save JD to resume for ATS check later
await ref.read(resumeNotifierProvider(widget.resumeId).notifier)
.updateTargetJD(_jdCtrl.text.trim());
// Run keyword match
final buf = StringBuffer();
final p = resume.sections['personal'] ?? {};
buf.writeln('\${p['summary'] ?? ''}');
for (final e in (resume.sections['experience'] as List? ?? []))
buf.writeln('\${e['description'] ?? ''}');
buf.writeln((resume.sections['skills'] as List? ?? []).join(', '));
final result = await ref.read(aiServiceProvider).matchJD(buf.toString(), _jdCtrl.text);
setState(() { _result = result; _loading = false; });
} catch (_) { setState(() => _loading = false); }
}

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Job Description Matcher')),
  body: Padding(padding: const EdgeInsets.all(16), child: Column(children: [
    TextField(controller: _jdCtrl, maxLines: 6,
      decoration: const InputDecoration(labelText: 'Paste Job Description here',
        border: OutlineInputBorder(), alignLabelWithHint: true)),
    const SizedBox(height: 12),
    SizedBox(width: double.infinity,
      child: ElevatedButton(onPressed: _loading ? null : _analyse,
        child: _loading ? const CircularProgressIndicator(color: Colors.white)
          : const Text('Analyse Match'))),
    if (_result != null) ...[
      const SizedBox(height: 20),
      LinearProgressIndicator(value: _result!.matchPercentage / 100,
        backgroundColor: Colors.red.shade100, color: Colors.green),
      const SizedBox(height: 6),
      Text('\${_result!.matchPercentage}% keyword match',
        style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 16),
        textAlign: TextAlign.center),
      const SizedBox(height: 16),
      Expanded(child: Row(crossAxisAlignment: CrossAxisAlignment.start, children: [
        Expanded(child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
          const Text('Matched', style: TextStyle(color: Colors.green, fontWeight:
            FontWeight.bold)),
          const SizedBox(height: 8),
          Expanded(child: ListView(children: _result!.matched.map((k) =>

```

```

Chip(label: Text(k, style: const TextStyle(fontSize: 11)),
backgroundColor: Colors.green.shade50)).toList()),
)),
const SizedBox(width: 12),
Expanded(child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
const Text('Missing', style: TextStyle(color: Colors.red, fontWeight: FontWeight.bold)),
const SizedBox(height: 8),
Expanded(child: ListView(children: _result!.missing.map((k) =>
ActionChip(label: Text(k, style: const TextStyle(fontSize: 11)),
backgroundColor: Colors.red.shade50,
onPressed: () { /* Add to skills */ })).toList()),
)),
)),
],
)),
);
}

```

21. Resume Preview Screen

lib/features/preview/resume_preview_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:printing/printing.dart';
import 'package:go_router/go_router.dart';
import '../providers/resume_provider.dart';
import '../services/pdf_service.dart';

class ResumePreviewScreen extends ConsumerWidget {
  final String resumeId;
  const ResumePreviewScreen({super.key, required this.resumeId});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final resumeAsync = ref.watch(resumeStreamProvider(resumeId));
    return Scaffold(
      appBar: AppBar(
        title: const Text('Preview'),
        actions: [
          IconButton(icon: const Icon(Icons.print),
            onPressed: () async {
              final resume = resumeAsync.value;
              if (resume != null) await PDFService().printResume(resume);
            },
          ),
          IconButton(icon: const Icon(Icons.share),
            onPressed: () { /* generate share link */ },
          ),
        ],
      ),
      body: resumeAsync.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => Center(child: Text('\$e')),
        data: (resume) => PdfPreview(
          build: (format) async {
            final file = await PDFService().generatePDF(resume);
            return file.readAsBytesSync();
          },
          allowPrinting: false,
          allowSharing: false,
          canChangePageFormat: false,
          pdfPreviewPageDecoration: BoxDecoration(
            border: Border.all(color: Colors.grey.shade300),
            boxShadow: [BoxShadow(color: Colors.black12, blurRadius: 8)],
          ),
        ),
      ),
    );
  }
}
```

```

),
),
bottomNavigationBar: SafeArea(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: ElevatedButton.icon(
      onPressed: () => context.push('/download/$resumeId'),
      icon: const Icon(Icons.download),
      label: const Text('Download for Free – Watch Ad'),
      style: ElevatedButton.styleFrom(
        minimumSize: const Size(double.infinity, 52)),
    ),
  ),
),
);
}
}

```

22. PDF Generation Service

lib/services/pdf_service.dart

```
import 'dart:io';
import 'package:flutter_html_to_pdf/flutter_html_to_pdf.dart';
import 'package:path_provider/path_provider.dart';
import 'package:printing/printing.dart';
import '../models/resume_model.dart';

class PDFService {
  static const Map<String,String> _colors = {
    'indigo':'#4F46E5','blue':'#2563EB','slate':'#475569',
    'green':'#059669','red':'#DC2626','purple':'#7C3AED',
    'orange':'#EA580C','black':'#111827',
  };

  String _buildHTML(ResumeModel r, String color) {
    final p = r.sections['personal'] ?? {};
    final exp = r.sections['experience'] as List? ?? [];
    final edu = r.sections['education'] as List? ?? [];
    final skills = r.sections['skills'] as List? ?? [];
    final certs = r.sections['certifications'] as List? ?? [];
    final projs = r.sections['projects'] as List? ?? [];

    String _bullets(String text) {
      if (text.isEmpty) return '';
      if (!text.contains('
')) return '<p style="margin:0">\$text</p>';
      final items = text.split('
').where((l) => l.trim().isNotEmpty)
        .map((l) => '<li>\${l.replaceFirst(RegExp(r"^[*-] "), "")}</li>').join();
      return '<ul style="margin:2px 0;padding-left:16px">\$items</ul>';
    }

    return '<!DOCTYPE html><html><head><meta charset="UTF-8">
<style>
/* ATS-SAFE: single-column, no tables, standard fonts, black body text */
* { margin:0;padding:0;box-sizing:border-box; }
body { font-family:Arial,Helvetica,sans-serif;font-size:11pt;line-height:1.4;
color:#000;max-width:210mm;padding:14mm 18mm; }
.name { font-size:22pt;font-weight:bold;color:\$color;margin-bottom:3px; }
.contact { font-size:9.5pt;color:#333;margin-bottom:14px; }
.contact span { margin-right:10px; }
h2 { font-size:12pt;font-weight:bold;color:\$color;text-transform:uppercase;
letter-spacing:.5px;border-bottom:1.5px solid \$color;
padding-bottom:2px;margin:13px 0 7px 0; }

```

```

.entry { margin-bottom:9px; }
.row { display:flex;justify-content:space-between;align-items:baseline; }
.title { font-weight:bold;font-size:11pt; }
.dates { font-size:10pt;color:#555; }
.sub { font-size:10pt;color:#333;margin-bottom:2px; }
.desc { font-size:10pt; }
.desc ul { padding-left:15px; }
.desc li { margin-bottom:1px; }
.skill { display:inline-block;background:#f0f0f0;border:1px solid #ddd;
padding:1px 6px;border-radius:3px;font-size:9.5pt;margin:2px; }
</style></head><body>
<div class="name">\${p['name']}</div>
<div class="contact">
\${p['email']!=null?'<span>\${p['email']}</span>':''}
\${p['phone']!=null?'<span>\${p['phone']}</span>':''}
\${p['location']!=null?'<span>\${p['location']}</span>':''}
\${p['linkedin']!=null?'<span>\${p['linkedin']}</span>':''}
\${p['portfolio']!=null?'<span>\${p['portfolio']}</span>':''}
</div>
\${(p['summary']??'').isEmpty?'<h2>Professional Summary</h2><p
style="font-size:10.5pt">\${p['summary']}</p>':''}
\${exp.isEmpty?'<h2>Work Experience</h2>'+exp.map((e)=>
'<div class="entry"><div class="row"><span class="title">\${e['title']}</span><span
class="dates">\${e['dates']}</span></div>'
'<div class="sub">\${e['company']}</div>\${(e['location']??'').isEmpty?' |
\${e['location']}</div>'
'<div class="desc">\${_bullets(e['description']??'')}</div></div>').join()':''}
\${edu.isEmpty?'<h2>Education</h2>'+edu.map((e)=>
'<div class="entry"><div class="row"><span class="title">\${e['degree']}</span><span
class="dates">\${e['year']}</span></div>'
'<div class="sub">\${e['institution']}</div>\${(e['gpa']??'').isEmpty?'<div
class="sub">GPA: \${e['gpa']}</div>':''}</div>').join()':''}
\${skills.isEmpty?'<h2>Skills</h2><div>'+skills.map((s)=>'<span
class="skill">\${s}</span>').join()+'</div>':''}
\${projs.isEmpty?'<h2>Projects</h2>'+projs.map((e)=>
'<div class="entry"><div class="row"><span class="title">\${e['name']}</span><span
class="dates">\${e['dates']}</span></div>'
'<div class="desc">\${_bullets(e['description']??'')}</div></div>').join()':''}
\${certs.isEmpty?'<h2>Certifications</h2><ul
style="padding-left:16px;font-size:10pt">+
certs.map((c)=>'<li>\${c['name']}</li> - \${c['issuer']}</li>
(\${c['year']??'')</li>').join()+'</ul>':''}
</body></html>''';
}

Future<File> generatePDF(ResumeModel resume) async {
final color = _colors[resume.colorTheme] ?? '#4F46E5';

```

```

final html = _buildHTML(resume, color);
final dir = await getApplicationDocumentsDirectory();
final name = resume.title.replaceAll(RegExp(r'^a-zA-Z0-9'), '_');
final result = await FlutterHtmlToPdf.convertFromHtmlContent(
html, dir.path, '\${name}_\${DateTime.now().millisecondsSinceEpoch}');
return File(result.path);
}

Future<void> printResume(ResumeModel resume) async {
final color = _colors[resume.colorTheme] ?? '#4F46E5';
final html = _buildHTML(resume, color);
await Printing.layoutPdf(
onLayout: (fmt) => Printing.convertHtml(format: fmt, html: html));
}
}

```


23. AdMob Rewarded Ads — Download Screen

lib/services/admob_service.dart

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:google_mobile_ads/google_mobile_ads.dart';

final adServiceProvider = Provider<AdMobService>((ref) =>
AdMobService()..loadRewardedAd());
final bannerAdProvider = Provider<Widget?>((ref) {
final svc = ref.watch(adServiceProvider);
return svc.buildBannerWidget();
});

class AdMobService {
  RewardedAd? _rewarded;
  BannerAd? _banner;
  InterstitialAd? _interstitial;
  bool _rewardedLoaded = false;
  int _attempts = 0;

  static String get _rewardedId => Platform.isAndroid
? (const bool.fromEnvironment('dart.vm.product')
? 'ca-app-pub-REAL_ANDROID_ID/REWARDED'
: 'ca-app-pub-3940256099942544/5224354917')
: (const bool.fromEnvironment('dart.vm.product')
? 'ca-app-pub-REAL_IOS_ID/REWARDED'
: 'ca-app-pub-3940256099942544/1712485313');

  static String get _bannerId => Platform.isAndroid
? 'ca-app-pub-3940256099942544/6300978111' // Test banner
: 'ca-app-pub-3940256099942544/2934735716';

  static String get _interstitialId => Platform.isAndroid
? 'ca-app-pub-3940256099942544/1033173712' // Test interstitial
: 'ca-app-pub-3940256099942544/4411468910';

  Future<void> loadRewardedAd() async {
    await RewardedAd.load(adUnitId: _rewardedId, request: const AdRequest(),
    rewardedAdLoadCallback: RewardedAdLoadCallback(
    onAdLoaded: (ad) { _rewarded = ad; _rewardedLoaded = true; _attempts = 0; },
    onAdFailedToLoad: (err) {
      _rewardedLoaded = false; _rewarded = null; _attempts++;
      if (_attempts < 3) Future.delayed(const Duration(seconds: 3), loadRewardedAd);
    }));
  }
}
```

```

Future<bool> showRewardedAd({required VoidCallback onRewarded,
required VoidCallback onFailed}) async {
  if (!_rewardedLoaded || _rewarded == null) { onFailed(); return false; }
  bool rewarded = false;
  _rewarded!.fullScreenContentCallback = FullScreenContentCallback(
    onAdDismissedFullScreenContent: (ad) {
      ad.dispose(); _rewarded = null; _rewardedLoaded = false;
      loadRewardedAd(); // Pre-load next
    },
    onAdFailedToShowFullScreenContent: (ad, err) {
      ad.dispose(); _rewarded = null; _rewardedLoaded = false; onFailed();
    });
  await _rewarded!.show(onUserEarnedReward: (_, __) { rewarded = true; onRewarded(); });
  return true;
}

Future<void> loadBannerAd() async {
  _banner = BannerAd(adUnitId: _bannerId, size: AdSize.banner,
    request: const AdRequest(), listener: BannerAdListener());
  await _banner!.load();
}

Widget? buildBannerWidget() {
  if (_banner == null) { loadBannerAd(); return null; }
  return SizedBox(height: _banner!.size.height.toDouble(),
    child: AdWidget(ad: _banner!));
}

Future<void> loadInterstitialAd() async {
  await InterstitialAd.load(adUnitId: _interstitialId, request: const AdRequest(),
    adLoadCallback: InterstitialAdLoadCallback(
      onAdLoaded: (ad) => _interstitial = ad,
      onAdFailedToLoad: (_) => _interstitial = null));
}

Future<void> showInterstitialAd() async {
  await _interstitial?.show();
  _interstitial = null;
  loadInterstitialAd();
}

bool get isRewardedReady => _rewardedLoaded && _rewarded != null;
}

```

lib/features/preview/download_screen.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

```

```

import '../services/admob_service.dart';
import '../services/pdf_service.dart';
import '../providers/resume_provider.dart';

class DownloadScreen extends ConsumerStatefulWidget {
  final String resumeId;
  const DownloadScreen({super.key, required this.resumeId});
  @override
  ConsumerState<DownloadScreen> createState() => _DLState();
}

class _DLState extends ConsumerState<DownloadScreen> {
  bool _loading = false;
  bool _done = false;
  String _status = 'Watch a short ad to download your resume completely FREE';

  @override
  void initState() {
    super.initState();
    ref.read(adServiceProvider).loadRewardedAd();
  }

  Future<void> _download() async {
    final adSvc = ref.read(adServiceProvider);
    if (!adSvc.isRewardedReady) {
      setState(() => _status = 'Loading ad, please wait...');
      await adSvc.loadRewardedAd();
      await Future.delayed(const Duration(seconds: 2));
      if (!adSvc.isRewardedReady) {
        setState(() => _status = 'Ad not available right now. Please try again.');
```

```

        return;
      }
      setState(() => _loading = true);
      await adSvc.showRewardedAd(
        onRewarded: () async {
          try {
            final resume = ref.read(resumeStreamProvider(widget.resumeId)).value!;
            final file = await PDFService().generatePDF(resume);
            await ref.read(resumeNotifierProvider(widget.resumeId).notifier).incrementDownload();
            setState(() { _done = true; _loading = false;
              _status = 'Resume saved! Check your Downloads folder.'; });
          } catch (e) { setState(() { _loading = false; _status = 'Error: \${e}'; }); }
        },
        onFailed: () => setState(() { _loading = false;
          _status = 'Please watch the complete ad to unlock download.'; })),
      );
    }
  }
}

```

```

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Download Resume')),
  body: Center(child: Padding(padding: const EdgeInsets.all(32), child: Column(
    mainAxisAlignment: MainAxisAlignment.center, children: [
      Icon(_done ? Icons.check_circle : Icons.play_circle_outline,
        size: 80, color: _done ? Colors.green : const Color(0xFF6366F1)),
      const SizedBox(height: 24),
      Text(_status, textAlign: TextAlign.center,
        style: const TextStyle(fontSize: 16)),
      const SizedBox(height: 32),
      if (!_done) _loading
        ? const Column(children: [CircularProgressIndicator(), SizedBox(height: 12),
          Text('Loading ad...')])
        : ElevatedButton.icon(onPressed: _download,
          icon: const Icon(Icons.play_arrow),
          label: const Text('Watch Ad & Download FREE'),
          style: ElevatedButton.styleFrom(
            minimumSize: const Size(280, 52))),
      const SizedBox(height: 16),
      const Text('FREE • No payment required • 15-30 second ad',
        style: TextStyle(color: Colors.grey, fontSize: 12)),
    ]))),
);
}

```

25. Cover Letter + 26. Job Tracker + 27. Pro Upgrade + 28. Settings

lib/features/cover_letter/cover_letter_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import '../services/ai_service.dart';
import '../services/firestore_service.dart';
import '../providers/resume_provider.dart';
import '../providers/auth_provider.dart';

class CoverLetterScreen extends ConsumerStatefulWidget {
  final String resumeId;
  const CoverLetterScreen({super.key, required this.resumeId});
  @override
  ConsumerState<CoverLetterScreen> createState() => _CLState();
}

class _CLState extends ConsumerState<CoverLetterScreen> {
  final _companyCtrl = TextEditingController();
  final _jdCtrl = TextEditingController();
  final _letterCtrl = TextEditingController();
  bool _loading = false;

  Future<void> _generate() async {
    setState(() => _loading = true);
    try {
      final resume = ref.read(resumeStreamProvider(widget.resumeId)).value;
      final uid = ref.read(authStateProvider).value?.uid;
      if (resume == null || uid == null) return;
      final p = resume.sections['personal'] ?? {};
      final buf = StringBuffer();
      buf.writeln('\${p['summary'] ?? ''}');
      for (final e in (resume.sections['experience'] as List? ?? []))
        buf.writeln('\${e['title']} at \${e['company']}: \${e['description']}');
      final letter = await ref.read(aiServiceProvider).generateCoverLetter(
        resumeText: buf.toString(), jd: _jdCtrl.text,
        company: _companyCtrl.text, name: p['name'] ?? '');
      _letterCtrl.text = letter;
      await ref.read/firestoreServiceProvider).saveCoverLetter(
        uid, widget.resumeId, letter, _companyCtrl.text);
    } finally { setState(() => _loading = false); }
  }

  @override
  Widget build(BuildContext context) => Scaffold(
```

```

appBar: AppBar(title: const Text('Cover Letter Builder')),
body: SingleChildScrollView(padding: const EdgeInsets.all(16), children: [
  TextField(controller: _companyCtrl,
  decoration: const InputDecoration(labelText: 'Company Name',
  border: OutlineInputBorder()),
  const SizedBox(height: 12),
  TextField(controller: _jdCtrl, maxLines: 4,
  decoration: const InputDecoration(labelText: 'Job Description (optional)',
  border: OutlineInputBorder(), alignLabelWithHint: true)),
  const SizedBox(height: 12),
  SizedBox(width: double.infinity,
  child: ElevatedButton(onPressed: _loading ? null : _generate,
  child: _loading ? const CircularProgressIndicator(color: Colors.white)
  : const Text('Generate Cover Letter with AI'))),
  if (_letterCtrl.text.isNotEmpty) ...[
    const SizedBox(height: 20),
    TextField(controller: _letterCtrl, maxLines: 20,
    decoration: const InputDecoration(labelText: 'Cover Letter (editable)',
    border: OutlineInputBorder(), alignLabelWithHint: true)),
    const SizedBox(height: 12),
    ElevatedButton.icon(onPressed: () => context.push('/download/\${widget.resumeId}'),
    icon: const Icon(Icons.download),
    label: const Text('Download FREE – Watch Ad')),
  ],
]),
);
}

```

lib/features/job_tracker/job_tracker_screen.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../services/firestore_service.dart';
import '../providers/auth_provider.dart';

const List<String> kStatuses = ['applied', 'interview', 'offer', 'rejected'];
const Map<String,Color> kStatusColors = {
  'applied': Color(0xFF6366F1), 'interview': Color(0xFFFF59E0B),
  'offer': Color(0xFF10B981), 'rejected': Color(0xFFEF4444)};

class JobTrackerScreen extends ConsumerWidget {
  const JobTrackerScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final uid = ref.watch(authStateProvider).value?.uid ?? '';
    final apps = ref.watch(applicationsStreamProvider(uid));
    return Scaffold(

```

```

appBar: AppBar(title: const Text('Job Tracker')),
body: apps.when(
loading: () => const Center(child: CircularProgressIndicator()),
error: (e, _) => Center(child: Text('\$e')),
data: (list) => SingleChildScrollView(scrollDirection: Axis.horizontal,
child: Row(crossAxisAlignment: CrossAxisAlignment.start,
children: kStatuses.map((status) {
final items = list.where((a) => a['status'] == status).toList();
return _KanbanColumn(uid: uid, status: status,
items: items, ref: ref);
}).toList()))),
floatingActionButton: FloatingActionButton(
onPressed: () => _showAddDialog(context, ref, uid),
child: const Icon(Icons.add)),
);
}

void _showAddDialog(BuildContext ctx, WidgetRef ref, String uid) {
final comp = TextEditingController(), role = TextEditingController();
showDialog(context: ctx, builder: (_) => AlertDialog(
title: const Text('Add Application'),
content: Column(mainAxisSize: MainAxisSize.min, children: [
TextField(controller: comp, decoration: const InputDecoration(labelText: 'Company')),
TextField(controller: role, decoration: const InputDecoration(labelText: 'Role')),
]),
actions: [
TextButton(onPressed: () => Navigator.pop(ctx), child: const Text('Cancel')),
ElevatedButton(onPressed: () {
ref.read(firebaseServiceProvider).addApplication(uid, {
'company': comp.text, 'role': role.text, 'status': 'applied', 'notes': ''});
Navigator.pop(ctx);
}, child: const Text('Add')),
],
));
}

class _KanbanColumn extends StatelessWidget {
final String uid, status;
final List<Map<String, dynamic>> items;
final WidgetRef ref;
const _KanbanColumn({required this.uid, required this.status,
required this.items, required this.ref});
@override
Widget build(BuildContext context) => Container(
width: 200, margin: const EdgeInsets.all(8),

```

```

child: Column(children: [
Container(width: double.infinity, padding: const EdgeInsets.all(10),
decoration: BoxDecoration(color: kStatusColors[status],
borderRadius: const BorderRadius.vertical(top: Radius.circular(8))),
child: Text(status.toUpperCase(),
style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bold))),
Container(color: Colors.grey.shade100, padding: const EdgeInsets.all(8),
child: Column(children: items.map((app) => Card(
margin: const EdgeInsets.only(bottom: 8),
child: Padding(padding: const EdgeInsets.all(10), child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
children: [
Text(app['company'] ?? '', style: const TextStyle(fontWeight: FontWeight.bold)),
Text(app['role'] ?? '', style: const TextStyle(color: Colors.grey, fontSize: 12)),
const SizedBox(height: 6),
Wrap(spacing: 4, children: kStatuses.where((s) => s != status).map((s) =>
GestureDetector(onTap: () => ref.read(firestoreServiceProvider)
.updateApplicationStatus(uid, app['id'], s),
child: Container(padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
decoration: BoxDecoration(color: kStatusColors[s]!.withOpacity(0.1),
borderRadius: BorderRadius.circular(4)),
child: Text('Move to \${s[0].toUpperCase()}\${s.substring(1)}',
style: TextStyle(fontSize: 9, color: kStatusColors[s])))).toList(),
])))),
).toList()),
]));
]);
}

```

lib/features/pro/pro_upgrade_screen.dart

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../services/payment_service.dart';
import '../providers/auth_provider.dart';

class ProUpgradeScreen extends ConsumerStatefulWidget {
  const ProUpgradeScreen({super.key});
  @override
  ConsumerState<ProUpgradeScreen> createState() => _ProState();
}

class _ProState extends ConsumerState<ProUpgradeScreen> {
  bool _annual = true;
  bool _loading = false;
  late PaymentService _paymentService;
  @override
  void initState() {

```



```

super.initState();

_paymentService = PaymentService()..init();
}

@override
void dispose() { _paymentService.dispose(); super.dispose(); }

Future<void> _subscribe() async {
  setState(() => _loading = true);
  _paymentService.startProSubscription(
    plan: _annual ? 'annual' : 'monthly',
    onSuccess: (paymentId) {
      setState(() => _loading = false);
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Pro activated!'), backgroundColor: Colors.green));
      Navigator.pop(context);
    },
    onFailure: (err) {
      setState(() => _loading = false);
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Payment failed: \$err'), backgroundColor: Colors.red));
    },
  );
}

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Upgrade to Pro')),
  body: SingleChildScrollView(padding: const EdgeInsets.all(20), children: [
    const Text('Go Pro', style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
      textAlign: TextAlign.center),
    const SizedBox(height: 8),
    const Text('Unlock all features. Cancel anytime.',
      style: TextStyle(color: Colors.grey), textAlign: TextAlign.center),
    const SizedBox(height: 24),
    // Pricing toggle
    Row(mainAxisAlignment: MainAxisAlignment.center, children: [
      const Text('Monthly'), const SizedBox(width: 8),
      Switch(value: _annual, onChanged: (v) => setState(() => _annual = v)),
      const SizedBox(width: 8), const Text('Annual'),
      if (_annual) Container(margin: const EdgeInsets.only(left: 6),
        padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
        decoration: BoxDecoration(color: Colors.green, borderRadius: BorderRadius.circular(10)),
        child: const Text('Save 15%', style: TextStyle(color: Colors.white, fontSize: 10))),
    ],
    const SizedBox(height: 8),
    Text(_annual ? 'Rs 999 / year (Rs 83/month)' : 'Rs 99 / month',

```

```

style: const TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
textAlign: TextAlign.center),
const SizedBox(height: 24),
// Feature list
...['Unlimited resumes (Free: 2 max)', 'Unlimited ATS score checks (Free: 3/day)',
'Cover letter builder (AI-powered)', 'DOCX export (Word document)',
'Job application tracker (Kanban)', 'Version history – restore any version',
'LinkedIn optimiser', 'No ads in the app UI',
'Priority customer support'].map((f) =>
ListTile(leading: const Icon(Icons.check_circle, color: Colors.green, size: 20),
title: Text(f), dense: true)),
const SizedBox(height: 24),
_loading ? const Center(child: CircularProgressIndicator())
: ElevatedButton(onPressed: _subscribe,
style: ElevatedButton.styleFrom(minimumSize: const Size(double.infinity, 52)),
child: Text('Subscribe \$_annual ? 'Rs 999/year' : 'Rs 99/month'})),
const SizedBox(height: 12),
const Text('Secure payment via Razorpay. Cancel anytime.',
style: TextStyle(color: Colors.grey, fontSize: 11),
textAlign: TextAlign.center),
]),
);
}

```

29. Onboarding Screen + 30. Payment Service (Razorpay + Stripe)

lib/features/onboarding/onboarding_screen.dart

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'package:shared_preferences/shared_preferences.dart';

class OnboardingScreen extends StatefulWidget {
  const OnboardingScreen({super.key});
  @override
  State<OnboardingScreen> createState() => _OnboardingState();
}

class _OnboardingState extends State<OnboardingScreen> {
  final _controller = PageController();
  int _page = 0;
  final _slides = [
    _Slide(icon: Icons.analytics_outlined, color: Color(0xFF6366F1),
      title: 'Beat the ATS', subtitle:
        '75% of resumes are rejected by ATS before a human ever reads them. '
        'We give your resume an ATS score out of 100 with exact fixes.'),
    _Slide(icon: Icons.auto_awesome, color: Color(0xFF10B981),
      title: 'AI Writes For You', subtitle:
        'Type a rough job duty – AI rewrites it as a strong, quantified '
        'bullet point. Generate your professional summary in seconds.'),
    _Slide(icon: Icons.download_done_outlined, color: Color(0xFFFF59E0B),
      title: 'Download for FREE', subtitle:
        'Watch a short 15-second ad and download your polished resume PDF '
        'completely free. No subscription needed. No credit card.'),
  ];

  Future<void> _finish() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('onboarding_done', true);
    if (mounted) context.go('/login');
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    body: SafeArea(child: Column(children: [
      Align(alignment: Alignment.topRight,
        child: TextButton(onPressed: _finish, child: const Text('Skip'))),
      Expanded(child: PageView.builder(
        controller: _controller,
        onPageChanged: (i) => setState(() => _page = i),
        itemCount: _slides.length,
```

```

itemBuilder: (_, i) => _slides[i].build()),
Row(mainAxisAlignment: MainAxisAlignment.center,
children: List.generate(_slides.length, (i) => AnimatedContainer(
duration: const Duration(milliseconds: 300), margin: const EdgeInsets.all(4),
width: _page == i ? 24 : 8, height: 8,
decoration: BoxDecoration(color: _page == i ? const Color(0xFF6366F1) : Colors.grey,
borderRadius: BorderRadius.circular(4)))),
const SizedBox(height: 20),
Padding(padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 16),
child: ElevatedButton(onPressed: _page < _slides.length - 1
? () => _controller.nextPage(duration: const Duration(milliseconds: 300),
curve: Curves.easeInOut)
: _finish,
style: ElevatedButton.styleFrom(minimumSize: const Size(double.infinity, 52)),
child: Text(_page < _slides.length - 1 ? 'Next' : 'Get Started'))),
])),
);
}

class _Slide {
final IconData icon; final Color color;
final String title, subtitle;
const _Slide({required this.icon, required this.color,
required this.title, required this.subtitle});
Widget build() => Padding(padding: const EdgeInsets.all(32), child: Column(
mainAxisAlignment: MainAxisAlignment.center, children: [
Icon(icon, size: 96, color: color),
const SizedBox(height: 32),
Text(title, style: const TextStyle(fontSize: 26, fontWeight: FontWeight.bold),
textAlign: TextAlign.center),
const SizedBox(height: 16),
Text(subtitle, style: const TextStyle(fontSize: 15, color: Colors.grey),
textAlign: TextAlign.center),
]));
}

```

lib/services/payment_service.dart

```

import 'package:flutter/material.dart';
import 'package:razorpay_flutter/razorpay_flutter.dart';
import 'package:flutter_stripe/flutter_stripe.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:firebase_auth/firebase_auth.dart';

class PaymentService {

```

[illegible]

[illegible]

31. Backend API — Node.js/Express (Full)

```
server.js
```

[illegible]

[illegible]


```
app.post('/api/verify-payment', auth, async (req, res) => {
  const { payment_id, order_id, signature } = req.body;
  const expected = crypto.createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
    .update(`${order_id}|${payment_id}`).digest('hex');
  if (expected !== signature)
    return res.status(400).json({ verified: false, error: 'Signature mismatch' });
  const days = req.body.plan === 'annual' ? 365 : 30;
  await db.collection('users').doc(req.user.uid).update({
    plan: 'pro',
    planExpiry: admin.firestore.Timestamp.fromDate(
      new Date(Date.now() + days * 86400000))
  });
  res.json({ verified: true });
});
```

```
// ■■ Stripe: Create Subscription
```

```
app.post('/api/stripe/create-subscription', auth, async (req, res) => {
  const { plan, email } = req.body;
  const priceId = plan === 'annual'
    ? process.env.STRIPE_ANNUAL_PRICE_ID
    : process.env.STRIPE_MONTHLY_PRICE_ID;
  try {
    let customer = await stripe.customers.list({ email, limit: 1 });
    const customerId = customer.data.length
      ? customer.data[0].id
      : (await stripe.customers.create({ email })).id;
    const sub = await stripe.subscriptions.create({
      customer: customerId, items: [{ price: priceId }],
      payment_behavior: 'default_incomplete',
      expand: ['latest_invoice.payment_intent'] });
    res.json({
      clientSecret: sub.latest_invoice.payment_intent.client_secret,
      subscriptionId: sub.id });
  } catch (e) { res.status(500).json({ error: e.message }); }
});
```

```
//   Share Link Generator
```

```
app.post('/api/share-link', auth, async (req, res) => {
  const { resumeId } = req.body;
  const token = crypto.randomBytes(16).toString('hex');
  const expiry = new Date(Date.now() + 30 * 86400000);
  await db.collection('share_links').doc(token).set({
    uid: req.user.uid, resumeId, expiry: admin.firestore.Timestamp.fromDate(expiry)
  });
});
```

```
res.json({ link: `https://your-domain.com/r/${token}`, expiry });  
});  
  
const PORT = process.env.PORT || 3000;  
app.listen(PORT, () => console.log(`ATS Resume Builder backend running on port  
${PORT}`));
```

32. Firestore Security Rules

firestore.rules

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    // Users can only read/write their own data
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    match /resumes/{resumeId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    match /coverLetters/{coverId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    match /applications/{appId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
  }

  // Templates are public read, only admin write
  match /templates/{templateId} {
    allow read: if true;
    allow write: if false; // Only via Admin SDK
  }

  // Rate limits: users can read their own, only backend writes
  match /rate_limits/{docId} {
    allow read: if request.auth != null && docId.matches(request.auth.uid + '_.*');
    allow write: if false; // Only backend writes via Admin SDK
  }

  // Share links: public read, authenticated write
  match /share_links/{token} {
    allow read: if true;
    allow write: if request.auth != null;
  }

  // Block all other paths
  match /{document=**} {
    allow read, write: if false;
  }
}
```

33. AndroidManifest.xml + Info.plist Setup

android/app/src/main/AndroidManifest.xml (key additions)

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
android:maxSdkVersion="28"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"
android:maxSdkVersion="32"/>

<application
android:label="ATS Resume Builder"
android:name="\${applicationName}"
android:icon="@mipmap/ic_launcher"
android:requestLegacyExternalStorage="true">

<!-- AdMob App ID - REQUIRED -->
<meta-data
android:name="com.google.android.gms.ads.APPLICATION_ID"
android:value="ca-app-pub-XXXXXXXXXXXXXXXX-YYYYYYYYYY"/>

<!-- Razorpay -->
<activity android:name="com.razorpay.CheckoutActivity"
android:exported="false" android:theme="@style/Checkout"/>

<activity android:name=".MainActivity"
android:exported="true"
android:launchMode="singleTop"
android:taskAffinity=""
android:theme="@style/LaunchTheme">
<intent-filter>
<action android:name="android.intent.action.MAIN"/>
<category android:name="android.intent.category.LAUNCHER"/>
</intent-filter>
</activity>
</application>
</manifest>
```

ios/Runner/Info.plist (key additions)

```
<!-- Add inside <dict> -->
<!-- AdMob App ID - REQUIRED -->
<key>GADApplicationIdentifier</key>
<string>ca-app-pub-XXXXXXXXXXXXXXXX-ZZZZZZZZZZ</string>

<!-- Required for AdMob on iOS 14+ -->
<key>SKAdNetworkItems</key>
<array>
```

```
<dict>
<key>SKAdNetworkIdentifier</key>
<string>cstr6suwn9.skadnetwork</string>
</dict>
</array>

<!-- NSUserTrackingUsageDescription for ATT prompt -->
<key>NSUserTrackingUsageDescription</key>
<string>We use this to show you relevant ads and measure their effectiveness.</string>

<!-- Required for file downloads -->
<key>UIFileSharingEnabled</key>
<true/>
<key>LSSupportsOpeningDocumentsInPlace</key>
<true/>
```

34. GitHub Actions CI/CD Workflow

`.github/workflows/deploy.yml`

```
name: Build and Deploy

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    name: Run Tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with: { flutter-version: '3.19.0', channel: 'stable' }
      - run: flutter pub get
      - run: flutter analyze
      - run: flutter test

  build-android:
    name: Build Android APK/AAB
    runs-on: ubuntu-latest
    needs: test
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with: { flutter-version: '3.19.0' }
      - uses: actions/setup-java@v3
        with: { distribution: temurin, java-version: '17' }
      - name: Decode keystore
        run: echo \${{ secrets.KEYSTORE_BASE64 }} | base64 --decode > android/app/keystore.jks
      - name: Create key.properties
        run: |
          echo "storePassword=\${{ secrets.STORE_PASSWORD }}" > android/key.properties
          echo "keyPassword=\${{ secrets.KEY_PASSWORD }}" >> android/key.properties
          echo "keyAlias=\${{ secrets.KEY_ALIAS }}" >> android/key.properties
          echo "storeFile=keystore.jks" >> android/key.properties
      - run: flutter pub get
      - run: flutter build appbundle --release
      - run: flutter build appbundle --release --dart-define=GEMINI_API_KEY=\${{ secrets.GEMINI_API_KEY }}
```

```

--dart-define=GROQ_API_KEY=\${{ secrets.GROQ_API_KEY }}
- uses: actions/upload-artifact@v3
with: { name: android-release, path: build/app/outputs/bundle/release/*.aab }
- name: Deploy to Play Store (Internal)
uses: r0adkll/upload-google-play@v1
with:
serviceAccountJsonPlainText: \${{ secrets.PLAY_STORE_JSON }}
packageName: com.yourcompany.ats_resume_builder
releaseFiles: build/app/outputs/bundle/release/*.aab
track: internal

build-ios:
name: Build iOS IPA
runs-on: macos-latest
needs: test
if: github.ref == 'refs/heads/main'
steps:
- uses: actions/checkout@v4
- uses: subosito/flutter-action@v2
with: { flutter-version: '3.19.0' }
- run: flutter pub get
- run: flutter build ios --release --no-codesign
--dart-define=GEMINI_API_KEY=\${{ secrets.GEMINI_API_KEY }}
--dart-define=GROQ_API_KEY=\${{ secrets.GROQ_API_KEY }}
- name: Build & Sign with Fastlane
run: cd ios && fastlane beta
env:
FASTLANE_APPLE_APPLICATION_SPECIFIC_PASSWORD: \${{ secrets.APP_SPECIFIC_PASSWORD }}
MATCH_PASSWORD: \${{ secrets.MATCH_PASSWORD }}

deploy-backend:
name: Deploy Backend to Render
runs-on: ubuntu-latest
needs: test
if: github.ref == 'refs/heads/main'
steps:
- name: Trigger Render Deploy
run: curl -X POST \${{ secrets.RENDER_DEPLOY_HOOK }}

```

35. Development Timeline + Cost Estimate

Phase	Weeks	Deliverables
Phase 1 — Setup & Auth	1–2	Flutter project, Firebase setup, Auth (Google + email), basic navigation, dashboard shell
Phase 2 — Resume Editor	3–4	All form sections, auto-save, Firestore integration, section reordering, multi-resume support
Phase 3 — Templates & PDF	5–6	Template system, live preview, PDF generation (text layer), ATS formatting enforcement
Phase 4 — AI Integration	7–8	Gemini + Groq fallback: bullet writer, summary generator, ATS score checker, JD keyword matcher
Phase 5 — Ad Integration	9	Google AdMob rewarded ads, banner + interstitial ads, download unlock flow
Phase 6 — Cover Letter & Polish	10	Cover letter builder, UI polish, animations, onboarding, error handling
Phase 7 — Beta & QA	11	Internal testing, ATS compatibility testing across 5 ATS systems, bug fixes
Phase 8 — Launch	12	Play Store + App Store submission, landing page, ASO

Service	Plan	Monthly Cost
Firebase Firestore	Free (Spark) — 1GB, 50k reads/day	FREE
Firebase Auth	Free — up to 10k MAU	FREE
Firebase Storage	Free — 5GB	FREE
Render Backend	Free tier — 512MB RAM	FREE
Gemini API	Free tier — 500 req/day	FREE
Groq API (Fallback)	Free tier — 14,400 req/day	FREE
Google AdMob	Revenue share	FREE
Google Play Developer	One-time Rs 1,750	One-time
Apple Developer	Rs 7,999/year	Rs 667/mo
Custom Domain	Rs 700–1,200/year	Rs 100/mo
TOTAL Year 1	—	~Rs 8,750 one-time + Rs 800/mo

36. Competitive Landscape + 37. Risks & Quick Reference

Competitor	Pricing	Download Model	ATS	Your Advantage
Rezi	\$29/month	Subscription paywall	Strong (23 checks)	Free via ad
Kickresume	\$179/year	Pay to download	Strong (GPT-4)	No payment barrier
Resume.io	\$2.95/7 days	Trial then paywall	Good	Free forever
Canva Resume	Free	Free download	Weak	Stronger ATS + AI
Novoresume	\$15.99/month	Pay to download	Good	Free via ad
Naukri Builder	Free (limited)	Free download	Basic	AI + ATS score
YOUR APP	Free + Rs 99/mo Pro	Watch ad (FREE)	Strong (24 checks)	All of the above

Risks & Mitigation

Risk	Mitigation
Ad revenue volatility	Add Pro subscription as second stream. Never depend solely on ads. Target Rs 99/mo converts 2–5% of users.
Gemini API limit changes	Groq fallback already implemented (14,400 req/day free). Budget \$90/mo for Gemini paid at scale.
ATS algorithm changes	Test templates against 5 real ATS systems quarterly (Workday, Greenhouse, Lever, iCIMS, Taleo).
User privacy (PII in AI)	Use paid Gemini tier (no data training). Clear privacy policy. Never store resume text on backend logs.
App Store policy	Rewarded ad model is explicitly allowed. Follow Google + Apple rewarded ad UX guidelines strictly.
Competition copying model	Build moat: better AI quality, more templates, B2B relationships. Speed to market is your primary advantage.

Quick Reference Summary

Item	Decision
Platform	Flutter (Dart) — Android + iOS + Web
State Management	Riverpod 2.x (compile-safe, code-generated providers)
Backend	Node.js + Express on Render (free tier → \$7/mo at scale)
Database	Firebase Firestore (Spark free tier)
Auth	Firebase Auth — Google Sign-In + email/password
AI Primary	Google Gemini 2.5 Flash — free 500 req/day
AI Fallback	Groq (Llama 3) — free 14,400 req/day
PDF Generation	flutter_html_to_pdf (client-side, ATS-safe text layer)
Rewarded Ads	Google AdMob + IronSource mediation (at 1k+ DAU)
Banner/Interstitial Ads	Google AdMob (passive supplementary revenue)
Payments India	Razorpay (Rs 99/mo or Rs 999/year Pro)
Payments Global	Stripe (\$4.99/mo or \$39.99/year Pro)
Download Model	Watch rewarded ad (15–30s) → unlock PDF download — ALWAYS FREE
Free Features	Build resume, ATS score (3/day), AI rewrite (5/day), 2 resumes, 10 templates
Pro Features	Unlimited everything, cover letters, DOCX export, job tracker, no ads
MVP Timeline	12 weeks solo / 6–8 weeks 2-person team
MVP Infra Cost	~Rs 0 first year (all free tiers) + Rs 8,750 store fees
Break-even	~500 downloads/day covers all costs
Revenue at 50k DAU	\$15,000+/month (ads alone) + Pro subscriptions

FINAL NOTE: Start with MVP Phases 1–5 (12 weeks), validate the watch-ad-to-download model with real users, then invest in Pro features and B2B. Every competitor charges money to download. You never do. That is your moat. Protect it with execution speed.